

02393 Programming in C++

Module 1:

An overview of C++

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- What is C++?
- A brief look at C++ programs
- Recommended C++ compiler and editor
- Lab time

What is C++

- C++ is a general-purpose programming language that extends C
 - Created by Bjarne Stroustrup (beginnings: 1979)
 - Adopted by the industry (circa 1990 onwards)
 - Now regulated by ISO C++ Standard Committee (C++98, C++03, . . . , C++20, . . .)
- Unique combinations of
 - High level features (e.g., object oriented and generic programming)
 - low-level features (manual memory management, memory pointers)

C++ compared to other programming languages

- C++ vs Python, Matlab, etc.
 - Statically typed, compiled, manual memory handling (no garbage collection)
- C++ vs. Java, C#, Visual Basic, etc.
 - Manual memory handling (no garbage collection)
- C++ vs C
 - Object-oriented, more advanced types
- C++ vs Rust
 - Object-oriented, less advanced types, more mature, bigger ecosystem

An overview of C++ programs

- The structure of a C++ program
 - #include directives, the main function, user-defined functions
- Simple input/output
 - cin, cout
- Variables, values and types
 - string, int, double, float, bool
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - if, while, for

A very simple C++ program

```
1  int main()  
2  {  
3      int myVar = 3;  
4  
5      if (myVar < 2) {  
6          return 0;  
7      } else {  
8          return 1;  
9      }  
10 }
```

A very simple C++ program

```
1  int main()
2  {
3      int myVar = 3;
4
5      if (myVar < 2) {
6          return 0;
7      } else {
8          return 1;
9      }
10 }
```

- main() function:
 - Must always exist
 - Will be invoked when the program is run
 - Will return an integer value
 - By convention, if the return value is equal to 0, it indicates that the program ran successfully
 - If not equal to 0, it indicates that an error occurred

A very simple C++ program

```
1  int main()
2  {
3      int myVar = 3;
4
5      if (myVar < 2) {
6          return 0;
7      } else {
8          return 1;
9      }
10 }
```

- Variable declaration:
 - An integer value named myVar is declared and initialized to 3
 - It is possible to declare variables without initializing them
 - In this case, the variable will contain a random value
 - Uninitialized variables can be accessed later on in the program
 - If not properly initialized, this can lead to nasty bugs
 - It is a good practice to initialize variables either while they are declared, or immediately after

A very simple C++ program

```
1  int main()  
2  {  
3      int myVar = 3;  
4  
5      if (myVar < 2) {  
6          return 0;  
7      } else {  
8          return 1;  
9      }  
10 }
```

- If/else statement:
 - Checks if myVar is lower than 2
 - If so, it executes the code inside the if part of the block
 - If not, it executes the code inside the else part of the block

A very simple C++ program

```
1  int main()
2  {
3      int myVar = 3;
4
5      if (myVar < 2) {
6          return 0;
7      } else {
8          return 1;
9      }
10 }
```

- Return statement:
 - Terminates the execution of main() by passing the specified value to the caller
 - In this case, the value is a constant, but it can also be a variable
 - For the main block, the caller is the operating system that runs our program

A classical beginner's program: hello world!

- The cout object represents the standard output (data that is shown as the text output of a program)
 - It can be used for our program to output messages

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7
8      cout << "Hello World!" << endl;
9
10     return 0;
11 }
```

A classical beginner's program: hello world!

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7
8      cout << "Hello World!" << endl;
9
10     return 0;
11 }
```

- The cout object represents the standard output (data that is shown as the text output of a program)
 - It can be used for our program to output messages
- To output a message, the “Hello World!” string is passed to cout via the << operator

A classical beginner's program: hello world!

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      cout << "Hello World!" << endl;
8
9      return 0;
10 }
11
```

- The cout object represents the standard output (data that is shown as the text output of a program)
 - It can be used for our program to write messages on screen
- To output a message, the “Hello World!” string is passed to cout via the << operator
- To write subsequent output on a new line, the endl object is passed to cout after the Hello World!” string
 - If endl is not passed, any subsequent output would be written on the same line!

A classical beginner's program: hello world!

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7
8      cout << "Hello World!" << endl;
9
10     return 0;
11 }
```

- The cout and endl objects are not part of the C++ standard constructs
 - Instead, they are part of the iostream library
- The #include statement tells the compiler that our program needs the iostream library to work

A classical beginner's program: hello world!

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7
8      cout << "Hello World!" << endl;
9
10     return 0;
11 }
```

- Library functions and objects with the same name may exist in different libraries
- To disambiguate them, namespaces are used
 - They work similarly to packages in other programming languages
- For the `iostream` library, `cout` and `endl` live inside the `std` namespace
- To tell the compiler in which namespace to look for them, one should use `std::cout`
- To save some typing, the `using namespace std` statement tells the compiler to use `std`

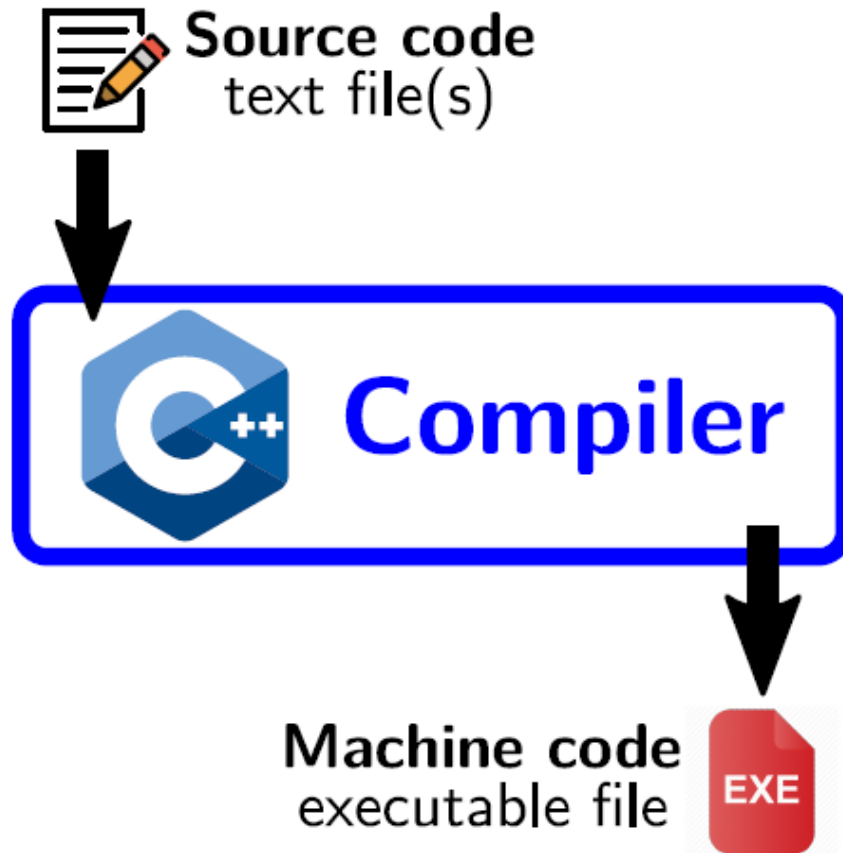
Live code example: a simple calculator

- I will now show C++ in action, by using an online C++ editor and compiler: <http://cpp.sh>
 - Try to follow along
 - If something is unclear, please raise your hand
 - For the next lectures, you will need to install a C++ compiler on your computer (more on this later)

Break

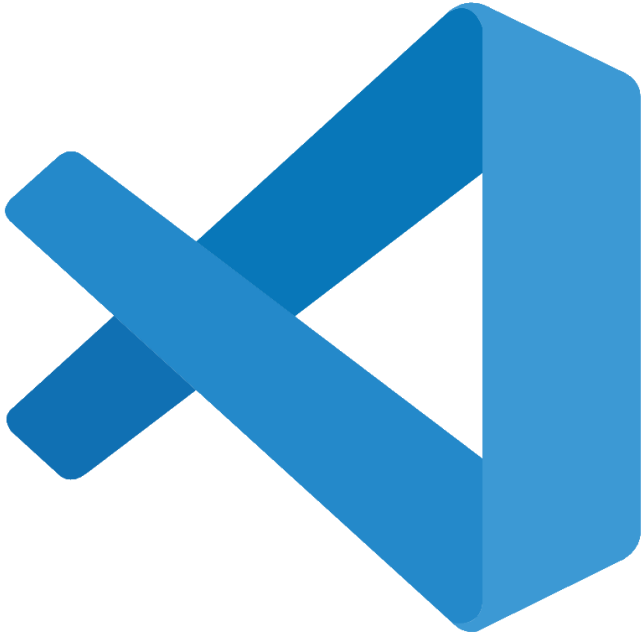
- We resume at 19:00

The compile process



- A C++ compiler is a program that:
 - reads source code from one or more text files
 - generates an executable file containing machine code
- You can use any text editor to write the source code
 - ...but maybe you want syntax highlighting at least
- There are many C++ compilers with various differences:
 - different compatibility with C++ language standards
 - different usage options
 - different error messages

Recommended for this course



- Microsoft VS Code (not Visual Studio) + GNU Compiler Collection (which includes the C++ compiler g++)
 - Both platforms are open source and exists for most desktop platforms
 - g++ is standard compliant and used on Autolab
- See instructions on Autolab on how to configure your environment

Lab time & homework

- Install Visual Studio Code and GCC (g++) on your computer
 - Follow the instructions on DTU Learn
 - If you have trouble, ask for help
 - Mac OS users: you may have a compiler called g++ that is not GCC. It should be OK. . .
- Try the first weekly assignment on your computer and on Autolab
 - The assignment description is available on DTU Learn
 - Goal: check your C++ installation and familiarize with Autolab
 - Don't worry if you don't get all exercises right :-)

DTU



02393 Programming in C++

Module 2:

Basic C++ and Data Types

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Compiling C++ programs
- Pointers
- Arrays
- More about data types
- Enumerations
- Structures
- Miscellaneous features
- Lab

A recap from the first lecture

- The structure of a C++ program
 - #include directives, the main function, user-defined functions
- Simple input/output
 - cin, cout
- Variables, values and types
 - string, int, double, float, bool
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - if, while, for

Compiling C++ programs

1. Edit the .cpp source code file (e.g., program.cpp) and save it
 - Microsoft Visual Studio Code is the recommended editor
2. Compile program.cpp producing the executable file program.exe
 - With GCC, the run the command: `g++ program.cpp -o program.exe`
 - With MacOS and older versions of Xcode tools, run the command:
`g++ program.cpp -o program.exe -std=c++17`
3. If you don't get compilation errors, run the resulting executable:
 - `./program.exe`

Checking the compiler output

```
$ g++ test.cpp -o test.exe
test.cpp: In function 'int main()':
test.cpp:8:29: error: 'miss' was not declared in this scope
   8 |         cout << "Hello " << miss << endl;
     |                               ^~~~~
```

- Variable “miss” was not declared

```
$ g++ test.cpp -o test.exe
test.cpp: In function 'int main()':
test.cpp:8:33: error: expected ';' before 'return'
   8 |         cout << "Hello " << endl
     |                                   ^
   9 |         return 0;
     |         ~~~~~
```

- A semicolon is missing

```
$ g++ test.cpp -o test.exe
test.cpp: In function 'int main()':
test.cpp:7:9: error: 'cout' was not declared in this scope
   7 |         cout << "Hello " << endl
     |         ^~~~~
test.cpp:3:1: note: 'std::cout' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
   2 | #include <string>
   +++ |+#include <iostream>
   3 |
test.cpp:7:29: error: 'endl' was not declared in this scope
   7 |         cout << "Hello " << endl
     |                               ^~~~~
test.cpp:3:1: note: 'std::endl' is defined in header '<iostream>'; did you forget to '#include <iostream>'?
   2 | #include <string>
   +++ |+#include <iostream>
   3 |
```

- iostream library was not included

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

- “Normal” variables contain a value

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8     " and points to value " << *p << endl;
```

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

Pointers

```

1  int x = 5;
2  int y = 42;
3  int *p = &y;
4  cout << " The value of p is " << p <<
5      " and points to value " << *p << endl;
6  p = &x;
7  cout << " The value of p is " << p <<
8      " and points to value " << *p << endl;

```

Address	Contents
0x1000	x = 5
0x1004	y → 42
0x1008	p → 0x1004

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address
- To access the contents of the memory address, one needs to prepend * to the pointer variable name

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << "The value of p is " << p <<
5     " and points to value " << *p << endl;
6 p = &x;
7 cout << "The value of p is " << p <<
8     " and points to value " << *p << endl;
```

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1004

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address
- To access the contents of the memory address, one needs to prepend * to the pointer variable name
- To get the memory address where the contents of a variable reside, one needs to prepend & to the variable name

Pointers

```
1 int x = 5;
2 int y = 42;
3 int *p = &y;
4 cout << " The value of p is " << p <<
5   " and points to value " << *p << endl;
6 p = &x;
7 cout << " The value of p is " << p <<
8   " and points to value " << *p << endl;
```

Address	Contents
0x1000	x = 5
0x1004	y = 42
0x1008	p = 0x1000

- “Normal” variables contain a value
- A variable of type pointer contains a memory address
 - The pointer type tells what type of value is stored in that address
- When accessing a pointer by its name, we access the memory address
- To access the contents of the memory address, one needs to prepend * to the pointer variable name
- To get the memory address where the contents of a variable reside, one needs to prepend & to the variable name
 - It can be used to assign that address to a pointer

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 double b[3];  
3  
4 cout << a[0] + 1 << endl;  
5 b[2] = 3.14;
```

- An array is a collection of values of a same type
- We can easily define arrays when their size is known at compile-time
 - We can initialize arrays when they are created

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 double b[3];  
3  
4 cout << a[0] + 1 << endl;  
5 b[2] = 3.14;
```

- An array is a collection of values of a same type
- We can easily define arrays when their size is known at compile-time
 - We can initialize arrays when they are created
 - We can defer the initialization later:
 - The array is created, but its contents are uninitialized

Arrays in C++

- To access array elements, we use indexing, starting with 0
 - Reading contents from the first element of array a

```
1  int a[] = {1, 2, 3};  
2  double b[3];  
3  
4  cout << a[0] + 1 << endl;  
5  b[2] = 3.14;
```

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 double b[3];  
3  
4 cout << a[0] + 1 << endl;  
5 b[2] = 3.14;
```

- To access array elements, we use indexing, starting with 0
 - Reading contents from the first element of array a
 - Writing contents to the third element of array b

Arrays in C++

```
1 int a[] = {1, 2, 3};  
2 a[3000] = 6;  
3 cout << "Hello " << a[3000] << endl;
```

- Arrays do not contain information on their size
 - You need to store it elsewhere
- You must pay attention not to index an array beyond its last element
 - If you don't, you will end up reading or writing memory areas which belong to other variables, or to other programs!
 - Therefore, the execution may lead to unspecified results or crashes!

Arrays in C++

```
1 int a[] = {100 , 101 , 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a <<
5     " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p <<
7     " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] <<
9     " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

Arrays in C++

```
1 int a[] = {100 , 101 , 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a << endl;
5     " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p << endl;
7     " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] << endl;
9     " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

Arrays in C++

```
1 int a[] = {100 , 101 , 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a <<
5   " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p <<
7   " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] <<
9   " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

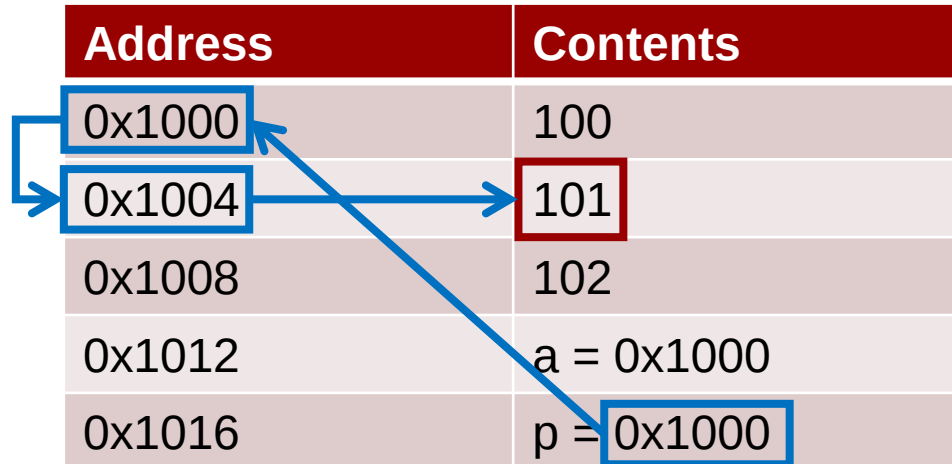
Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000

Arrays in C++

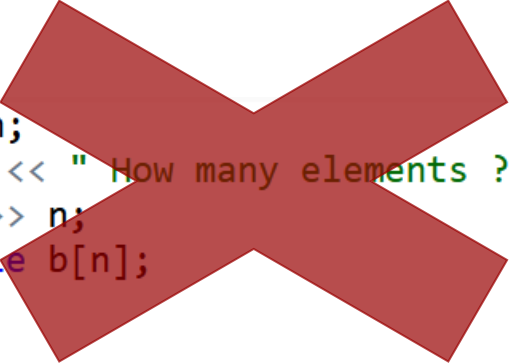
```
1 int a[] = {100, 101, 102};
2 int *p;
3 p = a;
4 cout << " The value of a is " << a <<
5     " and points to value " << a[0] << endl;
6 cout << " The value of p is " << p <<
7     " and points to value " << *p << endl;
8 cout << "p[0]= " << p[0] << " p[1]= " << p[1] <<
9     " p[2]= " << p[2] << endl;
```

- A variable of type array is just a pointer to the beginning of the array

Address	Contents
0x1000	100
0x1004	101
0x1008	102
0x1012	a = 0x1000
0x1016	p = 0x1000



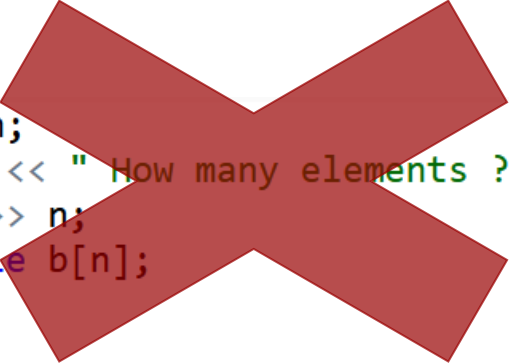
Arrays in C++



```
1 int n;  
2 cout << "How many elements?" << endl;  
3 cin >> n;  
4 double b[n];
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!

Arrays in C++

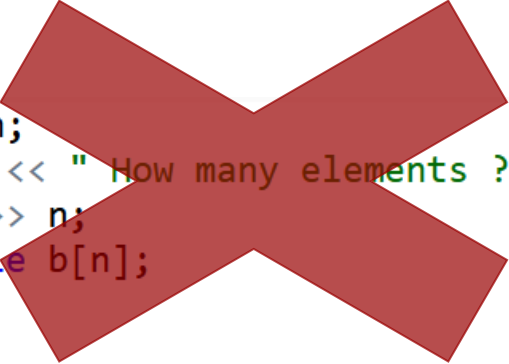


```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double b[n];
```

```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double *b = new double[n];  
5 // Use b as a normal array...  
6 delete[] b;  
-
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array

Arrays in C++

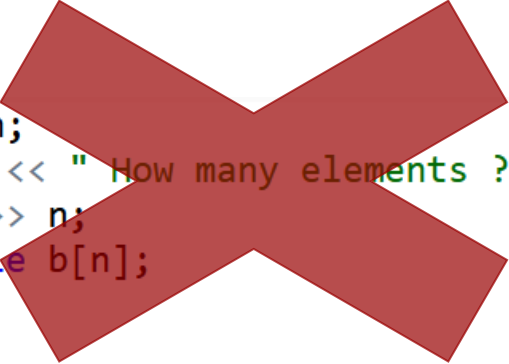


```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double b[n];
```

```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double *b = new double[n];  
5 // Use b as a normal array...  
6 delete[] b;  
-
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array
 - new allocates an array of size n

Arrays in C++



```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double b[n];
```

```
1 int n;  
2 cout << " How many elements ?" << endl;  
3 cin >> n;  
4 double *b = new double[n];  
5 // Use b as a normal array...  
6 delete[] b;
```

- What to do if the size of the array is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array
 - new allocates an array of size n
 - delete deallocates the memory area occupied by the array

Break

We resume at 19:30

Data types precision and conversions

- long double
- double
- float
- unsigned long
- long
- unsigned int
- int
- unsigned short
- short
- char

Most precise



Least precise

- C++ numeric data types differ in memory usage and precision
 - They all have minimum and maximum values
 - The compiler may perform automatic conversions toward the “largest” type
 - We can explicitly cast (convert) values across types (but be careful!)
 - floats and doubles are sometimes approximated, leading to rounding errors

Enumerations

```
1  enum Direction {NORTH, EAST, SOUTH, WEST};
2  Direction dir = NORTH;
3
4  void tellDirection(Direction dir) {
5  switch (dir) {
6      case NORTH:
7          cout << " North " << endl;
8          break;
9      case WEST:
10         cout << " West " << endl;
11         break;
12     case SOUTH:
13         cout << " South " << endl;
14         break;
15     case EAST:
16         cout << " East " << endl;
17         break;
18     default:
19         cout << " WARNING : invalid direction : " << dir << endl;
20     }
21 }
```

- An enumeration is a user-defined data type with values limited to a given finite set

Enumerations

```
1 enum Direction {NORTH, EAST, SOUTH, WEST};
2 Direction dir = NORTH;
3
4 void tellDirection(Direction dir) {
5     switch (dir) {
6         case NORTH:
7             cout << " North " << endl;
8             break;
9         case WEST:
10            cout << " West " << endl;
11            break;
12        case SOUTH:
13            cout << " South " << endl;
14            break;
15        case EAST:
16            cout << " East " << endl;
17            break;
18        default:
19            cout << " WARNING : invalid direction : " << dir << endl;
20    }
21 }
```

- An enumeration is a user-defined data type with values limited to a given finite set
- Very handy for representing symbolic information and using the switch statement

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2:  
9     e2.name = "Margaret";  
10    e2.surname = "Hamilton";  
11    e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized
- To access a member of a structure e1, we use the syntax e1.member

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized
- To access a member of a structure e1, we use the syntax e1.member
- If we have a pointer p to a structure, to access a member we can use:
 - (*p).member

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6 employee e1 = {"Bjarne", "Stroustrup", 100000};  
7  
8 employee e2;  
9 e2.name = "Margaret";  
10 e2.surname = "Hamilton";  
11 e2.salary = 500000;  
12  
13 employee *p = &e1;  
14 cout << "The employee name is: " << p->name << endl;  
15 cout << "The employee surname is: " << (*p).surname << endl;
```

- A structure is a user-defined data type that combines values of different types
 - The variables internal to the structure are called members
- We can initialize structures when they are created
- We can defer the initialization later:
 - The structure is created, but its contents are uninitialized
- To access a member of a structure e1, we use the syntax e1.member
- If we have a pointer p to a structure, to access a member we can use:
 - (*p).member
 - p->member

Structures (a.k.a. records)

```
1 struct employee {  
2     string name;  
3     string surname;  
4     unsigned int salary;  
5 };  
6  
7 employee employees[2];  
8  
9 employees[0] = {" John ", "Doe", 50000};  
10  
11 employees[1].name = "Margaret";  
12 employees[1].surname = "Hamilton";  
13 employees[1].salary = 500000;
```

- Being a (user-defined) data type, we can declare arrays of structures
- Handy for representing multiple records (e.g., an employee registry, an inventory, etc.).

Miscellaneous features

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable

```
1 #define MAXIMUM 10
2
3 int main () {
4     for (int i = 0; i < MAXIMUM; i++) {
5         string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6         cout << "The number " << i << " is " << oddOrEven << endl;
7     }
8     return 0;
9 }
```

Miscellaneous features

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable
 - Whenever `MAXIMUM` is used in the code, the compiler replaces it with 10
 - It works like find&replace
 - Handy to make the code parametric

```
1  #define MAXIMUM 10
2
3  int main () {
4      for (int i = 0; i < MAXIMUM; i++) {
5          string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6          cout << "The number " << i << " is " << oddOrEven << endl;
7      }
8      return 0;
9  }
```

Miscellaneous features

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable
 - Whenever `MAXIMUM` is used in the code, the compiler replaces it with 10
 - It works like find&replace
 - Handy to make the code parametric
- The `++` operator increments a numeric variable of 1

```
1  #define MAXIMUM 10
2
3  int main () {
4      for (int i = 0; i < MAXIMUM; i++) {
5          string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6          cout << "The number " << i << " is " << oddOrEven << endl;
7      }
8      return 0;
9  }
```

Miscellaneous features

```
1 #define MAXIMUM 10
2
3 int main () {
4     for (int i = 0; i < MAXIMUM; i++) {
5         string oddOrEven = ((i % 2) == 0) ? "even" : "odd";
6         cout << "The number " << i << " is " << oddOrEven << endl;
7     }
8     return 0;
9 }
```

- The preprocessor directive `#define` assigns 10 to the symbol `MAXIMUM`
 - `MAXIMUM` is NOT a variable
 - Whenever `MAXIMUM` is used in the code, the compiler replaces it with 10
 - It works like find&replace
 - Handy to make the code parametric
- The `++` operator increments a numeric variable of 1
- Conditional expressions are a compact way to express binary decisions:
 - If `i` modulo 2 is zero, “even” is assigned to `oddOrEven`
 - If not, “odd” is assigned to `oddOrEven`

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 2
 - Ask questions if something is unclear (including on Assignment 1)

DTU



02393 Programming in C++

Module 3:

Memory Management

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

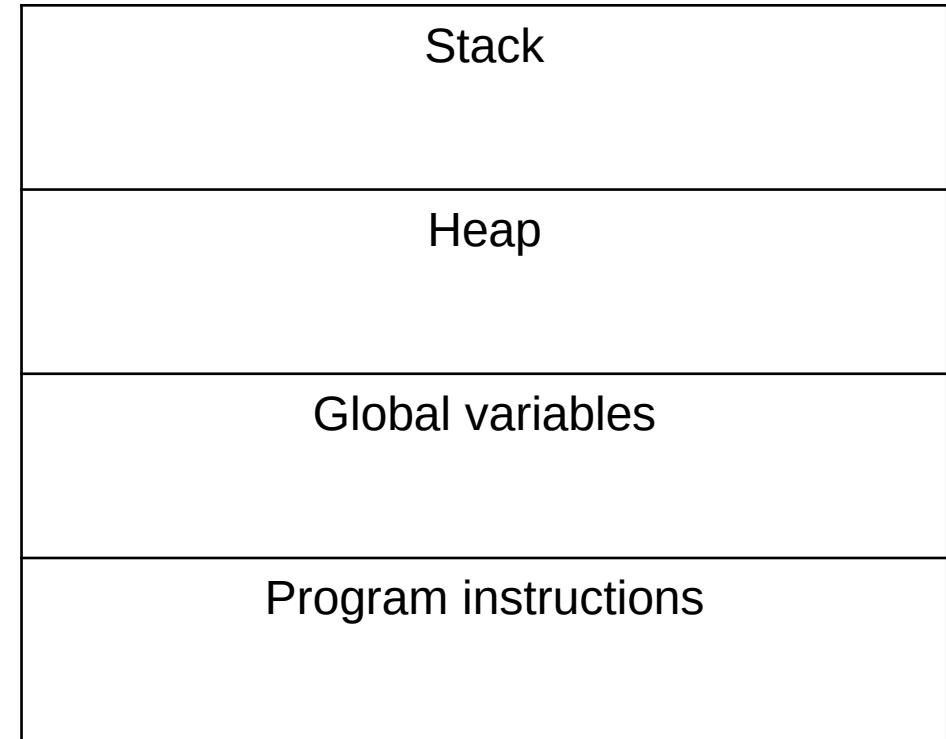
- Recap
- Stack vs heap
- Pointers vs references
- Bidimensional arrays
- Lab

A recap from the first lecture

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays, pointers, `enum`, `struct`
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - `if`, `while`, `for`, `switch`

(simplified) Memory map of a C++ program

- Stack: stores function parameters and local variables
- Heap: manual memory management with new and delete
- Global variables: accessible from anywhere in the program
- Program instructions: immutable



The function stack

```
1 int increment(int x) {  
2     int r = x + 1;  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int b = increment(a);  
9     return 0;  
10 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = ???

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

The function stack

```
1 int increment(int x) {  
2     int r = x + 1;  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int b = increment(a);  
9     return 0;  
10 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = ???
increment()	0x1012	x = 42
	0x1016	r = ???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted

The function stack

```
1 int increment(int x) {  
2     int r = x + 1;  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int b = increment(a);  
9     return 0;  
10 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = ???
increment()	0x1012	x = 42
	0x1016	r = 43

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

The function stack

```

1  int increment(int x) {
2      int r = x + 1;
3      return r;
4  }
5
6  int main() {
7      int a = 42;
8      int b = increment(a);
9      return 0;
10 }
```

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 43
increment()	0x1012	???
	0x1016	???

The function stack

```
1 int* makeArray() {  
2     int r[] = {55, 66};  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int *p = makeArray();  
9     p[0]++;  
10    return 0;  
11 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```
1 int* makeArray() {  
2     int r[] = {55, 66};  
3     return r;  
4 }  
5  
6 int main() {  
7     int a = 42;  
8     int *p = makeArray();  
9     p[0]++;  
10    return 0;  
11 }
```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???
makeArray()	0x1012	r[0] = 55
	0x1016	r[1] = 66
	0x1020	r = 0x1012

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1 int* makeArray() {
2     int r[] = {55, 66};
3     return r;
4 }
5
6 int main() {
7     int a = 42;
8     int *p = makeArray();
9     p[0]++;
10    return 0;
11 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x1012
makeArray()	0x1012	???
	0x1016	???
	0x1020	???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1 int* makeArray() {
2     int r[] = {55, 66};
3     return r;
4 }
5
6 int main() {
7     int a = 42;
8     int *p = makeArray();
9     p[0]++;
10    return 0;
11 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x1012
makeArray()	0x1012	???
	0x1016	???
	0x1020	???

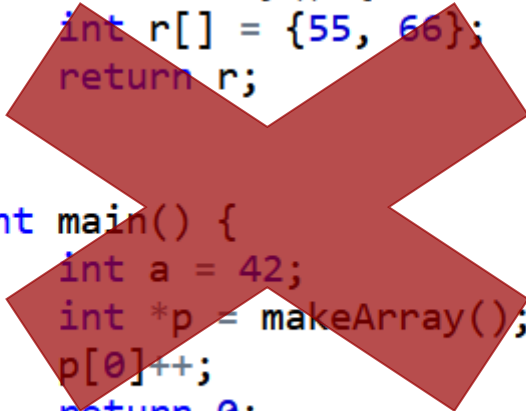
- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1 int* makeArray() {
2     int r[] = {55, 66};
3     return r;
4 }
5
6 int main() {
7     int a = 42;
8     int *p = makeArray();
9     p[0]++;
10    return 0;
11 }

```



Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x1012
makeArray()	0x1012	???
	0x1016	???
	0x1020	???



- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**

The function stack

```

1  int* makeArray() {
2      int *r = new int[2];
3      r[0] = 55; r[1] = 66;
4      return r;
5  }
6
7  int main() {
8      int a = 42;
9      int *p = makeArray();
10     p[0]++;
11     delete[] p;
12     return 0;
13 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

then the information will be remembered instead of deleted, as in the function stack

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	???
0x2004	???

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???
makeArray()	0x1012	r = 0x2000

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	r[0] = 55
0x2004	r[1] = 66

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = ???
makeArray()	0x1012	r = 0x2000

- Holds local variables and function parameters
- When a function *f* is invoked:
 - The input parameters are copied in *f*'s stack (call-by-value)
 - The return value of *f* is copied in the caller's stack
 - After *f* returns, *f*'s stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	r[0] = 55
0x2004	r[1] = 66

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x2000
makeArray()	0x1012	???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

The function stack

```

1 int* makeArray() {
2     int *r = new int[2];
3     r[0] = 55; r[1] = 66;
4     return r;
5 }
6
7 int main() {
8     int a = 42;
9     int *p = makeArray();
10    p[0]++;
11    delete[] p;
12    return 0;
13 }

```

Memory heap:

Address	Contents
0x2000	r[0] = 56
0x2004	r[1] = 66

remember to
free the memory from the heap with delete
(or else it's stuck here "forever")

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	p = 0x2000
makeArray()	0x1012	???

- Holds local variables and function parameters
- When a function f is invoked:
 - The input parameters are copied in f's stack (call-by-value)
 - The return value of f is copied in the caller's stack
 - After f returns, f's stack is automatically deleted
- Since arrays are pointers, the value of an array variable is a memory address
 - **You cannot return an array allocated on the stack**
 - You can return an array by allocating it on the heap instead

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 1
	0x1016	y = 42
	0x1020	tmp = 1

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 42
	0x1016	y = 42
	0x1020	tmp = 1

Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 42
	0x1016	y = 1
	0x1020	tmp = 1

Pointers vs references

```

1 void swap(int x, int y) {
2     int tmp = x;
3     x = y;
4     y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(a, b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	???
	0x1016	???
	0x1020	???



Pointers vs references

```
1 void swap(int x, int y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters

Pointers vs references

```
1 void swap(int *x, int *y) {  
2     int tmp = *x;  
3     *x = *y;  
4     *y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(&a, &b)  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b)
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 1
	0x1004	b = 42
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b)
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 42
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 1
swap()	0x1012	x = 0x1000
	0x1016	y = 0x1004
	0x1020	tmp = 1

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```

1 void swap(int *x, int *y) {
2     int tmp = *x;
3     *x = *y;
4     *y = tmp;
5 }
6
7 int main() {
8     int a = 1, b = 42;
9     cout << "a=" << a << ", b=" << b << endl;
10    swap(&a, &b);
11    cout << "a=" << a << ", b=" << b << endl;
12 }

```

Function stack	Address	Contents
main()	0x1000	a = 42
	0x1004	b = 1
swap()	0x1012	???
	0x1016	???
	0x1020	???

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can solve this problem with pointers
 - The input parameters contain memory addresses
 - Memory addresses won't be changed
 - We will swap the contents instead
 - When calling the function, we must pass the memory address of the variables with &

Pointers vs references

```
1 void swap(int &x, int &y) {  
2     int tmp = x;  
3     x = y;  
4     y = tmp;  
5 }  
6  
7 int main() {  
8     int a = 1, b = 42;  
9     cout << "a=" << a << ", b=" << b << endl;  
10    swap(a, b);  
11    cout << "a=" << a << ", b=" << b << endl;  
12 }
```

- How to write a function that swaps the values of two given variables?
 - We can return only one value
 - We cannot alter the contents of the input parameters
- We can also solve the problem with references
 - A reference is an alias to a variable
 - References cannot be null
 - References cannot point to a new variable
 - References must be initialized when created

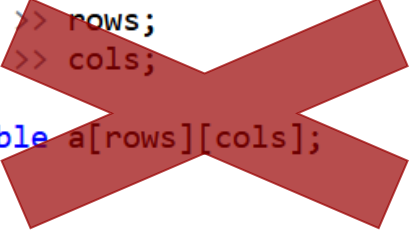
Bidimensional arrays

```
1 int a[2][2] = {{1, 2}, {3, 4}};  
2 double b[2][2];  
3  
4 cout << a[0][0] + a[1][1] << endl;  
5 b[1][0] = 3.14;
```

- We can easily define 2-dimensional arrays when their size is known at compile-time
- To access array elements, we use indexing, starting with 0
- You must pay attention not to index rows/columns beyond their last element
 - If you don't, you will end up reading or writing memory areas which belong to other variables, or to other programs!
 - Therefore, the execution may lead to unspecified results or crashes!

Bidimensional arrays

```
1 int rows, cols;
2
3 cout << "How many rows and columns?" << endl;
4 cin >> rows;
5 cin >> cols;
6
7 double a[rows][cols];
```



- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!

Bidimensional arrays

```
1 int main(){
2     int rows, cols;
3     cout << "How many rows & cols?";
4     cin >> rows; cin >> cols;
5
6     double **a = new double*[rows];
7     for (int i = 0; i < rows; i++) {
8         a[i] = new double[cols];
9     }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }
```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

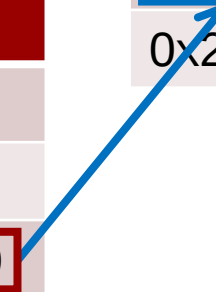
- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000

Address	Contents
0x2000	???
0x2004	???



Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0

Address	Contents
0x2000	0x3000
0x2004	???
0x3000	???
0x3004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	???
0x3004	???
0x4000	???
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0
0x1020	j = 0

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	???
0x4000	???
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensionl array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0
0x1020	j = 1

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	0.0
0x4000	???
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers
Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1
0x1020	j = 0

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	0.0
0x4000	0.0
0x4004	???

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1
0x1020	j = 1

Address	Contents
0x2000	0x3000
0x2004	0x4000
0x3000	0.0
0x3004	0.0
0x4000	0.0
0x4004	0.0

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensionanl array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 0

Address	Contents
0x2000	???
0x2004	0x4000
0x4000	0.0
0x4004	0.0

Bidimensional arrays

```

1  int main(){
2      int rows, cols;
3      cout << "How many rows & cols?";
4      cin >> rows; cin >> cols;
5
6      double **a = new double*[rows];
7      for (int i = 0; i < rows; i++) {
8          a[i] = new double[cols];
9      }
10
11     for (int i = 0; i < rows; i++) {
12         for (int j = 0; j < cols; j++) {
13             a[i][j] = 0.0;
14         }
15     }
16
17     for (int i = 0; i < rows; i++) {
18         delete[] a[i];
19     }
20     delete[] a;
21     return 0;
22 }

```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

Memory heap:

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = 0x2000
0x1016	i = 1

Address	Contents
0x2000	???
0x2004	???



Bidimensional arrays

```
1 int main(){
2     int rows, cols;
3     cout << "How many rows & cols?";
4     cin >> rows; cin >> cols;
5
6     double **a = new double*[rows];
7     for (int i = 0; i < rows; i++) {
8         a[i] = new double[cols];
9     }
10
11    for (int i = 0; i < rows; i++) {
12        for (int j = 0; j < cols; j++) {
13            a[i][j] = 0.0;
14        }
15    }
16
17    for (int i = 0; i < rows; i++) {
18        delete[] a[i];
19    }
20    delete[] a;
21    return 0;
22 }
```

- How do we create a 2-dimensional array whose size is only known at runtime?
- One may try this code, but **this is not standard C++** and may not work!
- Instead, one needs to manually allocate and deallocate the array with pointers to pointers

main() stack:

Address	Contents
0x1000	rows = 2
0x1004	cols = 2
0x1012	a = ???

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 3
 - Ask questions if something is unclear (including previous assignments)
- Next Tuesday (LAB DAY on 24 September)
 - There will be **no new topics**, just programming practice.
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 4
 - Ask questions if something is unclear (including Assignments 1 and 2)

DTU



02393 Programming in C++

Module 5: Libraries and Interfaces

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

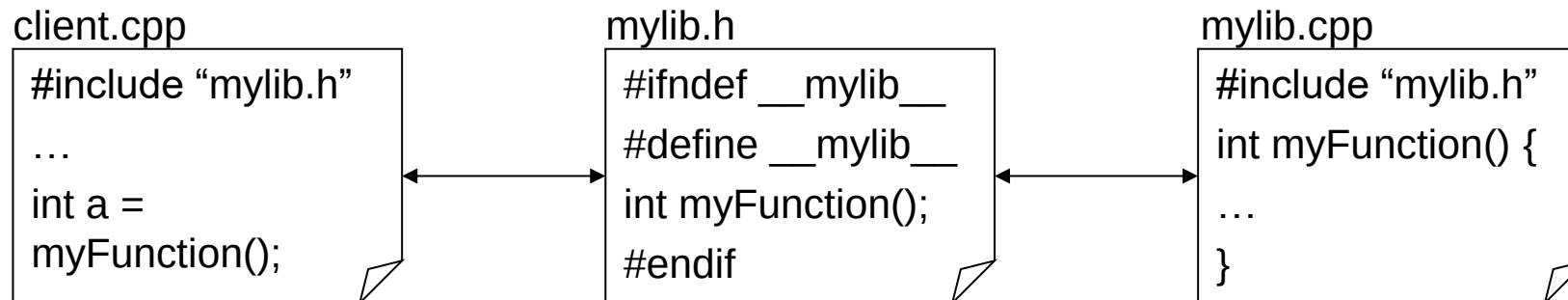
- Recap
- Libraries and interfaces
- The C++ Standard Template Library
- STL Vectors
- Standard I/O and file streams
- STL Strings
- Lab

A recap from the last lectures lecture

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - `if`, `while`, `for`, `switch`

Libraries and Interfaces

- A library is a collection of ready-to-use type definitions and functions
- A library interface is a “contract” between a library and its users (i.e., us programmers):
 - It provides information for using the library
 - It hides the implementation details
- The interface of a library is a header file (example: `iostream.h`) that only contains:
 - Function prototypes (not implementations!)
 - Type definitions
- The implementation of a library is written in one or more `.cpp` files



Libraries and Interfaces

```
1 #include <iostream>
2 #include "mylib.h"
```

- The `#include` directive is used to specify which libraries are used by a client program

Libraries and Interfaces

```
1 #include <iostream>  
2 #include "mylib.h"
```

- The `#include` directive is used to specify which libraries are used by a client program
 - If the library is part of the STL (and of the compiler), its name of the library is put inside angle brackets

Libraries and Interfaces

```
1 #include <iostream>
2 #include "mylib.h"
```

- The `#include` directive is used to specify which libraries are used by a client program
 - If the library is part of the STL (and of the compiler), its name of the library is put inside angle brackets
 - If the library is NOT part of the STL (e.g., it is created by the developer), the name of its header file is put inside quotes, and it must include the `.h` extension

Libraries and Interfaces

#ifndef: check if this symbol is already defined somewhere

--> if not defined, we define the symbol (__mylib__)

#when the compiler tries to import the library again,

it finds out that __mylib__ is already defined (exchange mylib with library name)

mylib.h

```
1  #ifndef __mylib__
2  #define __mylib__
3
4  int myFunction();
5
6  #endif
```

- To avoid a library to be included multiple times in a program, the #ifndef...#endif precompiler directives can be used in the header file.
- #ifndef checks if the symbol __mylib__ is already defined
 - If mylib has not been included, then __mylib__ is not defined. Thus, mylib code is included. Also, __mylib__ is defined.
 - If mylib has already been included, then __mylib__ is defined. Thus, the code block until #endif is ignored.

Libraries and Interfaces

- To compile a program that uses libraries not part of the STL, one must pass to the compiler the name of all .cpp files
 - E.g., `g++ client.cpp mylib.cpp -o client.exe`

The C++ Standard Template Library (STL)

- STL is a C++ library of container classes and algorithms
- Containers are collections of elements. Examples:
 - Unordered collections: set, mset
 - Array-like collections: vector, list, array (not the built-in arrays we already know!)
 - Other ordered collections: queue, stack
 - Dictionaries: map, multimap
- It is important to know how to deal with STL containers and choose the right one
 - More than one class of containers may do the job...
 - ...but some may do the job better (e.g., faster)
 - Before writing new code, it may be worth checking whether the STL can help!

STL Vector

- Arrays are fundamental data types in many programming languages, but in C++. . .
 - They are difficult/impossible to resize
 - Insertion and deletion can be difficult and slow
 - You have to keep track of their actual size
 - You have to be careful to index within the array bounds
 - You have to manually handle their allocation on the stack or on the heap
- The STL vector class solves all these problems!

STL Vector

- To use STL vectors, you need to include their interface (header file): `#include <vector>`
- The type `vector<X>` is a container of elements of base type `X`
 - `vector<int>` is a vector whose elements are integers
 - `vector<double>` is a vector whose elements are doubles
 - `vector<vector<int>>` is a vector whose elements are vectors of integers
 - ...and so on **vector of vectors!**

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

10

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

10

20

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

10	20	30
----	----	----

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

10	20	30	40
----	----	----	----

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1
- begin() returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers

STL Vector

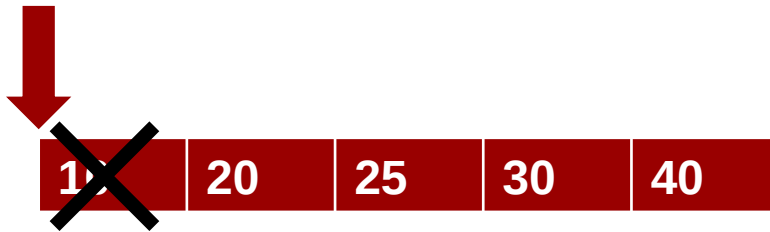
```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1
- begin() returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- insert(it, y) adds y before the element at the position specified by the iterator it
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



- `push_back(x)` adds `x` at the end of the vector
 - The size of the vector increases by 1
- `begin()` returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- `insert(it, y)` adds `y` before the element at the position specified by the iterator `it`
 - The size of the vector increases by 1
- `erase(it)` removes the element at the position specified by the iterator `it`
 - The size of the vector decreases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

20	25	30	40
----	----	----	----

- `push_back(x)` adds `x` at the end of the vector
 - The size of the vector increases by 1
- `begin()` returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- `insert(it, y)` adds `y` before the element at the position specified by the iterator `it`
 - The size of the vector increases by 1
- `erase(it)` removes the element at the position specified by the iterator `it`
 - The size of the vector decreases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



20	25	30	35
----	----	----	----

- `push_back(x)` adds `x` at the end of the vector
 - The size of the vector increases by 1
- `begin()` returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- `insert(it, y)` adds `y` before the element at the position specified by the iterator `it`
 - The size of the vector increases by 1
- `erase(it)` removes the element at the position specified by the iterator `it`
 - The size of the vector decreases by 1
- The content of a vector is indexed and can be accessed with the same syntax as arrays

Iterating through vector elements

```
1 for (int i = 0; i < vec.size(); i++) {  
2     cout << vec[i] << " ";  
3 }
```

- Array-style

```
1 vector<int>::iterator it;  
2 for (it = vec.begin(); it != vec.end(); it++) {  
3     cout << *it << " ";  
4 }
```

- Using iterators

```
1 for (auto e : vec) {  
2     cout << e << " ";  
3 }
```

- Modern style (range-based loop)

STL Vector and memory allocation

```
1 vector<int> makeVector() {  
2     vector<int> result;  
3     // ...  
4     return result;  
5 }  
6  
7 int main () {  
8     vector<int> vec = makeVector();  
9     // 'vec' is used here  
10    return 0;  
11 }
```

- Does this code work? Could it make the program crash?

STL Vector and memory allocation

```
1 vector<int> makeVector() {  
2     vector<int> result;  
3     // ...  
4     return result;  
5 }  
6  
7 int main () {  
8     vector<int> vec = makeVector();  
9     // 'vec' is used here  
10    return 0;  
11 }
```

- The vector internally stores its data in a dynamically allocated array (using new)
 - Hence, this internal array resides on the heap (not on the stack!)
- Some internal information of the vector (pointer to array, size) is kept on the stack
 - Such pointer and size are copied to the caller, inside vec, when makeVector() returns
 - Since the internal array survives on the heap, the code above works!
- When main() ends, vec is destroyed and it automatically deletes its internal array
 - Hence, no need to manually free the memory

STL Vector and memory allocation

```
1 void extendVector(vector<int> v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- Does this code change vec?

STL Vector and memory allocation

```
1 void extendVector(vector<int> v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)

STL Vector and memory allocation

use & because we don't want to create a copy of the vector, but a copy of the pointer

```
1 void extendVector(vector<int> &v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)
- If we want to modify vec, we can use references

STL Vector and memory allocation

```
1 void extendVector(const vector<int> &v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)
- If we want to modify vec, we can use references
- If we want to ensure that the vector is not modified, we can use the const keyword
 - The function treats the argument as a reference to a constant

STL Vector and memory allocation

```
1 void extendVector(const vector<int> &v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)
- If we want to modify vec, we can use references
- If we want to ensure that the vector is not modified, we can use the const keyword
 - The function treats the argument as a reference to a constant
 - If the code in the function tries to change vec, a compile error is produced

STL Vector and memory allocation

- Memory handling in STL vectors makes life easier
 - We can often avoid working with pointers, new and delete!
- Other STL **containers** (set, map, stack, ...) have the same convenient memory handling
- ...but what is going on behind the scenes in these containers?
 - We will see in the upcoming lectures on **object-oriented programming** in C++

Standard I/O and file streams

- STL includes the I/O library `iostream`, based on `streams`
 - The `cout` stream writes output to the console with its `insertion` operator `<<`
 - The `cin` stream takes input from console with its `extraction` operator `>>`
- STL also includes a `stream-based library` for file I/O, called `fstream`
 - `ofstream` objects write output to a file with their `insertion` operator `<<`
 - `ifstream` objects take input from a file with their `extraction` operator `>>`

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt")
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt
 - fail() tells the program if something went wrong when manipulating a file

Standard I/O and file streams

```
1 #include <fstream>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     ifstream infile;
7     ofstream outfile("output.txt");
8
9     infile.open("input.txt");
10    if (infile.fail()) {
11        cout << " ERROR : cannot open the file !" << endl;
12    }
13
14    string x;
15    infile >> x;
16    outfile << "I have read the string : " << x << endl;
17
18    infile.close();
19    outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt
 - fail() tells the program if something went wrong when manipulating a file
- Reading/writing data from/to the open files

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt
 - fail() tells the program if something went wrong when manipulating a file
- Reading/writing data from/to the open files
- Closing files when no longer needed
 - This happens automatically when the stream goes out of scope

STL Strings

- In C++, strings are natively represented as arrays of chars with last element 0
 - They have all disadvantages of arrays (difficult to resize, risk of out-of-bound access...)
- The STL `<string>` header file provides the string type that makes life much easier
 - We have already used it!
- Operations on STL strings:
 - Assignment using `=` (makes a new copy)
 - Comparison using `<`, `==`, `>=`, ... (by alphabetical ordering)
 - Concatenation using `+`

STL Strings

```
1 #include <string>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     string str1("Hello World");
7     string str2 = "Hello world";
8
9     cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- String creation

STL Strings

```
1 #include <string>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     string str1("Hello World");
7     string str2 = "Hello world";
8
9     cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- String creation
- Checking if a string is empty
 - True if empty
 - False otherwise

STL Strings

```
1  #include <string>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str1("Hello World");
7      string str2 = "Hello world";
8
9      cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- String creation
- Checking if a string is empty
 - True if empty
 - False otherwise
- Getting the length of a string
- ...but what is going on behind the scenes in STL strings?
 - We will see in the upcoming lectures on object-oriented programming in C++

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 5
 - Ask questions if something is unclear (including previous assignments)

DTU



02393 Programming in C++

Module 6: Classes and Objects

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Object-Oriented Programming
- Classes in C++
- Abstract Data Types
- Lab

A recap from the last lecture

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - `if`, `while`, `for`, `switch`

The “++” in C++

- So far, we have seen C programming with few elements of C++
 - string
 - cin/cout
 - references
 - vector
- C++ extends C with two key features:
 - Object-Oriented Programming (OOP) (today)
 - Templates for generic programming (next lecture)

Object-Oriented Programming

- We want to implement a library to handle bank accounts:
 - Each account will contain the following information
 - Name
 - Account number
 - Amount
 - Transferring money from one account to another

Object-Oriented Programming

- We can implement that library with struct and functions, however
 - The amount can be changed anywhere in our program
 - We want only the library code to change it
- Object-Oriented programming addresses this limitation

Object-Oriented Programming

- Procedural programming (what we did so far) focuses on actions:
 - Functions are the main elements of a program
 - Data exist primarily to support functions
- Object-Oriented programming focuses on data:
 - Data types and operations to manipulate them are grouped in classes
 - Classes are the main elements of a program
 - To represent data, classes are instantiated into objects

Classes in C++

- A class is similar to a struct, but its members can be:
 - Variables

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)

Classes in C++

```
1 class BankAccount {  
2  
3     private  
4         int number;  
5         string owner;  
6         double balance;  
7  
8     public:  
9         BankAccount(int number, string owner, double balance);  
10        ~BankAccount();  
11  
12        double getBalance() const;  
13        int getNumber() const;  
14        const string getOwner() const;  
15        void print() const;  
16        bool transferMoney(BankAccount &to, double value);  
17        bool withdraw(double value);  
18        bool deposit(double value);  
19    };  
20  
21    double BankAccount::getBalance() const {  
22        return this->balance;  
23    }  
24  
25    ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new
 - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new
 - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class consists in:
 - Class definition:
 - Attributes
 - Method prototypes
 - Can be moved to a library header file

Classes in C++

```
1 class BankAccount {  
2  
3 private:  
4     int number;  
5     string owner;  
6     double balance;  
7  
8 public:  
9     BankAccount(int number, string owner, double balance);  
10    ~BankAccount();  
11  
12    double getBalance() const;  
13    int getNumber() const;  
14    const string getOwner() const;  
15    void print() const;  
16    bool transferMoney(BankAccount &to, double value);  
17    bool withdraw(double value);  
18    bool deposit(double value);  
19 };  
20  
21 double BankAccount::getBalance() const {  
22     return this->balance;  
23 }  
24  
25 ...
```

- A class consists in:
 - Class definition:
 - Attributes
 - Method prototypes
 - Can be moved to a library header file
 - Class implementation:
 - Methods implementation

Using classes in C++

```
1  BankAccount accA(1, "Alice", 1000);  
2  BankAccount accB(2, "Bob", 500);  
3  
4  accA.print();  
5  accB.print();  
6  
7  accA.transferMoney(accB, 10);  
8  accA.print();  
9  accB.print();
```

- Instantiating an object
 - The constructor is called

Using classes in C++

```
1  BankAccount accA(1, "Alice", 1000);
2  BankAccount accB(2, "Bob", 500);
3
4  accA.print();
5  accB.print();
6
7  accA.transferMoney(accB, 10);
8  accA.print();
9  accB.print();
```

- Instantiating an object
 - The constructor is called
- Invoking a method for an object

Using classes in C++

```
1  #include <string>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str1("Hello World");
7      string str2 = "Hello world";
8
9      cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- Instantiating an object
 - The constructor is called
- Invoking a method for an object
- We have already seen all this during lecture 5.
 - E.g., when using strings

Redefining assignment operator

```
1 BankAccount accA(1, "alice", 1000);
2 BankAccount accB(2, "Bob", 500);
3
4 accB = accA;
5
6
7 BankAccount & BankAccount::operator=(BankAccount &ba){
8     this->owner = ba.owner;
9     return *this;
10 }
```

- By default, when object a is assigned to b, the content of a's attributes are copied over b's attributes (memberwise copy)
 - If a member is a pointer, only the address will be copied!
- To implement a different behaviour, we can redefine the assignment operator
 - this is the object to the left of the assignment
 - The input parameter is the object to the right of the assignment
 - Both are called by reference

Abstract Data Types

- Use C++ encapsulation to write code that abstracts from implementation details
 - Specify allowed operations on an ADT, by making them public
 - Hide everything else, by making it private
 - Instances of an ADT can only be constructed and used via public operations
- Programs that use a well-designed ADT do not need to be changed when the ADT's (private) implementation details are changed
- To show this principle, we will implement our own vector class, with the same behavior as the STL one
- Other examples are available in `lecture06-live-coding.zip` archive:
 - Matrix
 - Dictionary/Map
 - Bag

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 6
 - Ask questions if something is unclear (including previous assignments)

DTU



02393 Programming in C++

Module 7: Templates

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Templates in C++
- Survey
- Lab

A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators)
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`

Recap on Object-Oriented Programming

- Object-Oriented programming focuses on data:
 - Data types and operations to manipulate them are grouped in classes
 - Classes are the main elements of a program
 - To represent data, classes are instantiated into objects
- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new
 - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete
 - Assignment: customizes the behavior of the = operator.

Recap on Abstract Data Types

- Use C++ encapsulation to write code that abstracts from implementation details
 - Specify allowed operations on an ADT, by making them public
 - Hide everything else, by making it private
 - Instances of an ADT can only be constructed and used via public operations
- Programs that use a well-designed ADT do not need to be changed when the ADT's (private) implementation details are changed

The “++” in C++

- So far, we have seen C programming with few elements of C++
 - string
 - cin/cout
 - references
 - vector
- C++ extends C with two key features:
 - Object-Oriented Programming (OOP) (previous lecture)
 - Templates for generic programming (today)

Extending MyVector

- Last time, we implemented our own vector class MyVector
- However, objects of class MyVector can only contain int values
 - What if we need a MyVector of doubles or booleans?
- We need to make our code independent from the data types
- One solution would be to create a copy of MyVector code for each data type and adapt it accordingly
 - Lots of duplicated code, which is not maintainable
 - If a bug is discovered, all variants of MyVector need to be inspected
 - If a new feature is introduced, it must be done for each variant of MyVector

Introducing templates

- Templates: a key feature of C++ enabling generic programming
- Using templates, we can write code generic wrt. some element (types, numbers, etc.)
 - Payoffs: write less duplicated code and simplify maintenance
- The C++ Standard Library provides many facilities based on templates
 - e.g., containers like vector, set, map, etc.

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:

Function templates

```
1  template<class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:
 - We indicate that the function is a template

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:
 - We indicate that the function is a template
 - We specify T as a generic type

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:
 - We indicate that the function is a template
 - We specify T as a generic type
 - We replace data types for parameters and/or return value with T

Function templates

```
1  template <class T>
2  T maxF(T a, T b) {
3      if (a < b) {
4          return b;
5      } else {
6          return a;
7      }
8  };
9
10 int main(){
11     int x = maxF<int>(2, 3);
12     double y = maxF<double>(1.2, 3.5);
13 }
```

- To turn a function into a function template:
 - We indicate that the function is a template
 - We specify T as a generic type
 - We replace data types for parameters and/or return value with T
- To instantiate a function template, we specify what data type T will be

Function templates

```
1 struct BankAccount {
2     int number;
3     string owner;
4     double amount;
5 };
6
7 template <class T>
8 T maxF(T a, T b) {
9     if (a < b) {
10         return b;
11     } else {
12         return a;
13     }
14 };
15
16 template <>
17 BankAccount maxF(BankAccount a, BankAccount b) {
18     if (a.amount < b.amount) {
19         return b;
20     } else {
21         return a;
22     }
23 };
24
25 int main(){
26     int x = maxF<int>(2, 3);
27     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
29 }
30
```

- Some instances of the function template may not make sense and/or may not compile

Function templates

```
1 struct BankAccount {  
2     int number;  
3     string owner;  
4     double amount;  
5 };  
6  
7 template <class T>  
8 T maxF(T a, T b) {  
9     if (a < b) {  
10         return b;  
11     } else {  
12         return a;  
13     }  
14 };  
15  
16 template <>  
17 BankAccount maxF(BankAccount a, BankAccount b) {  
18     if (a.amount < b.amount) {  
19         return b;  
20     } else {  
21         return a;  
22     }  
23 };  
24  
25 int main(){  
26     int x = maxF<int>(2, 3);  
27     double y = maxF<double>(1.2, 3.5);  
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});  
29 }  
30
```

- Some instances of the function template may not make sense and/or may not compile
- For these specific cases, we refine the template

Function templates

```
1 struct BankAccount {
2     int number;
3     string owner;
4     double amount;
5 };
6
7 template <class T>
8 T maxF(T a, T b) {
9     if (a < b) {
10         return b;
11     } else {
12         return a;
13     }
14 };
15
16 template <>
17 BankAccount maxF(BankAccount a, BankAccount b) {
18     if (a.amount < b.amount) {
19         return b;
20     } else {
21         return a;
22     }
23 };
24
25 int main(){
26     int x = maxF<int>(2, 3);
27     double y = maxF<double>(1.2, 3.5);
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
29 }
30
```

- Some instances of the function template may not make sense and/or may not compile
- For these specific cases, we refine the template
 - We indicate that the function is a template

Function templates

```
1 struct BankAccount {
2     int number;
3     string owner;
4     double amount;
5 };
6
7 template <class T>
8 T maxF(T a, T b) {
9     if (a < b) {
10         return b;
11     } else {
12         return a;
13     }
14 };
15
16 // Specialization for BankAccount
17 BankAccount maxF(BankAccount a, BankAccount b) {
18     if (a.amount < b.amount) {
19         return b;
20     } else {
21         return a;
22     }
23 };
24
25 int main(){
26     int x = maxF<int>(2, 3);
27     double y = maxF<double>(1.2, 3.5);
28     BankAccount z = maxF<BankAccount>({1, "Alice", 1000}, {2, "Bob", 500});
29 }
30
```

- Some instances of the function template may not make sense and/or may not compile
- For these specific cases, we refine the template
 - We indicate that the function is a template
 - We specify data types for parameters and/or return value, as we did so far

Class templates

```
1  template <class A, class B>
2  class Pair {
3      private :
4          A a;
5          B b;
6      public:
7          A getA();
8          Pair(A a,B b);
9  };
10
11  template <class A, class B>
12  Pair<A, B>::Pair(A a, B b){
13      this->a = a;
14      this->b = b;
15  }
16  template <class A, class B>
17  A Pair<A, B>::getA(){
18      return this->a;
19  }
20
21  int main(){
22      Pair<int, string> p(10, "foo");
23      cout << p.getA();
24  }
```

- Templates can also be used to define generic classes

Class templates

```
1  template <class A, class B>
2  class Pair {
3      private :
4          A a;
5          B b;
6      public:
7          A getA();
8          Pair(A a,B b);
9  };
10
11  template <class A, class B>
12  Pair<A, B>::Pair(A a, B b){
13      this->a = a;
14      this->b = b;
15  }
16  template <class A, class B>
17  A Pair<A, B>::getA(){
18      return this->a;
19  }
20
21  int main(){
22      Pair<int, string> p(10, "foo");
23      cout << p.getA();
24  }
```

- Templates can also be used to define generic classes
 - When implementing methods, we need to specify that they are also templates

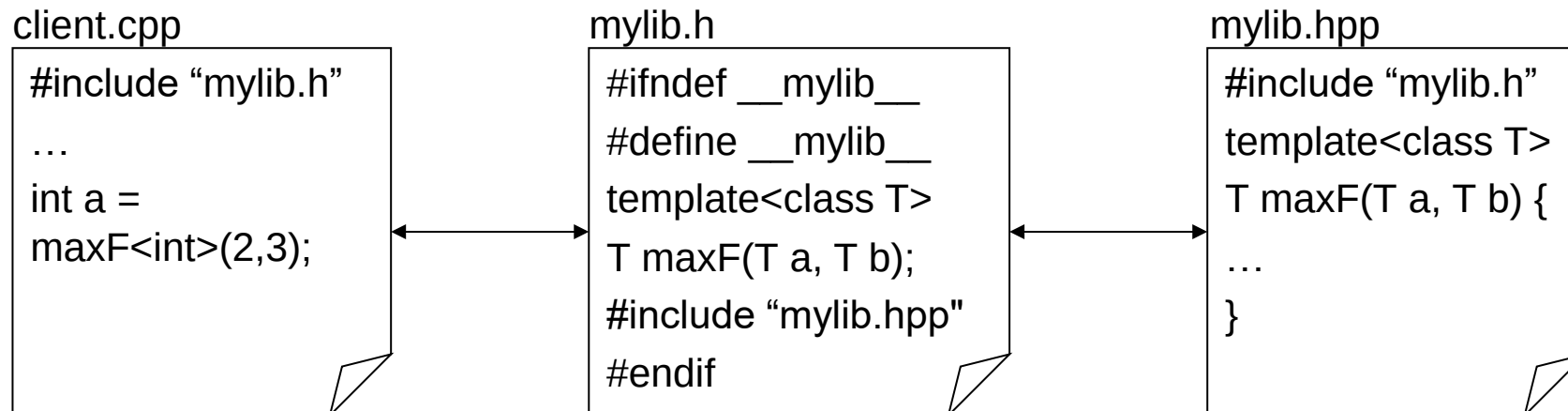
Class templates

```
1  template <class A, class B>
2  class Pair {
3      private :
4          A a;
5          B b;
6      public:
7          A getA();
8          Pair(A a,B b);
9  };
10
11 template <class A, class B>
12 Pair<A, B>::Pair(A a, B b){
13     this->a = a;
14     this->b = b;
15 }
16 template <class A, class B>
17 A Pair<A, B>::getA(){
18     return this->a;
19 }
20
21 int main(){
22     Pair<int, string> p(10, "foo");
23     cout << p.getA();
24 }
```

- Templates can also be used to define generic classes
 - When implementing methods, we need to specify that they are also templates
 - To instantiate a class template, we specify the data types

Templates in libraries

- In previous lectures we have seen that for code reusability we should have:
 - Interfaces (type declarations, function prototypes) in header files (.h)
 - Implementations (function and method definitions) in separate code files (.cpp)
- Template function prototypes go in a header .h file (as expected)
- Template function implementations are a code skeleton (not executable) and can go:
 - In a header .h file (works but it is messy)
 - In a template implementation file .hpp included in the library's main header file



Survey

- Please take part in a brief anonymous midterm survey
- You can find it on <https://evaluating.dtu.dk/>

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 7
 - The deadline is for Tuesday after autumn break
 - Ask questions if something is unclear (including previous assignments)

DTU



02393 Programming in C++

Module 8: Inheritance

© **Giovanni Meroni**

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Inheritance
- Derived classes
- Abstract classes
- Lab

A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`

Recap on Object-Oriented Programming

- Object-Oriented programming focuses on data:
 - Data types and operations to manipulate them are grouped in classes
 - Classes are the main elements of a program
 - To represent data, classes are instantiated into objects
- A class is similar to a struct, but its members can be:
 - Variables
 - Methods (a bit like functions)
- Class members can be public or private
 - Users of a class can only access public members (data encapsulation)
- Classes can have some special methods:
 - Constructor: called when an object is created, either statically, or dynamically using new
 - Destructor: called when an object is destroyed, either statically by exiting a scope, or dynamically using delete
 - Assignment: customizes the behavior of the = operator.

Recap on Abstract Data Types

- Use C++ encapsulation to write code that abstracts from implementation details
 - Specify allowed operations on an ADT, by making them public
 - Hide everything else, by making it private
 - Instances of an ADT can only be constructed and used via public operations
- Programs that use a well-designed ADT do not need to be changed when the ADT's (private) implementation details are changed
- We can use templates to make our code generic and reusable (e.g., for containers)

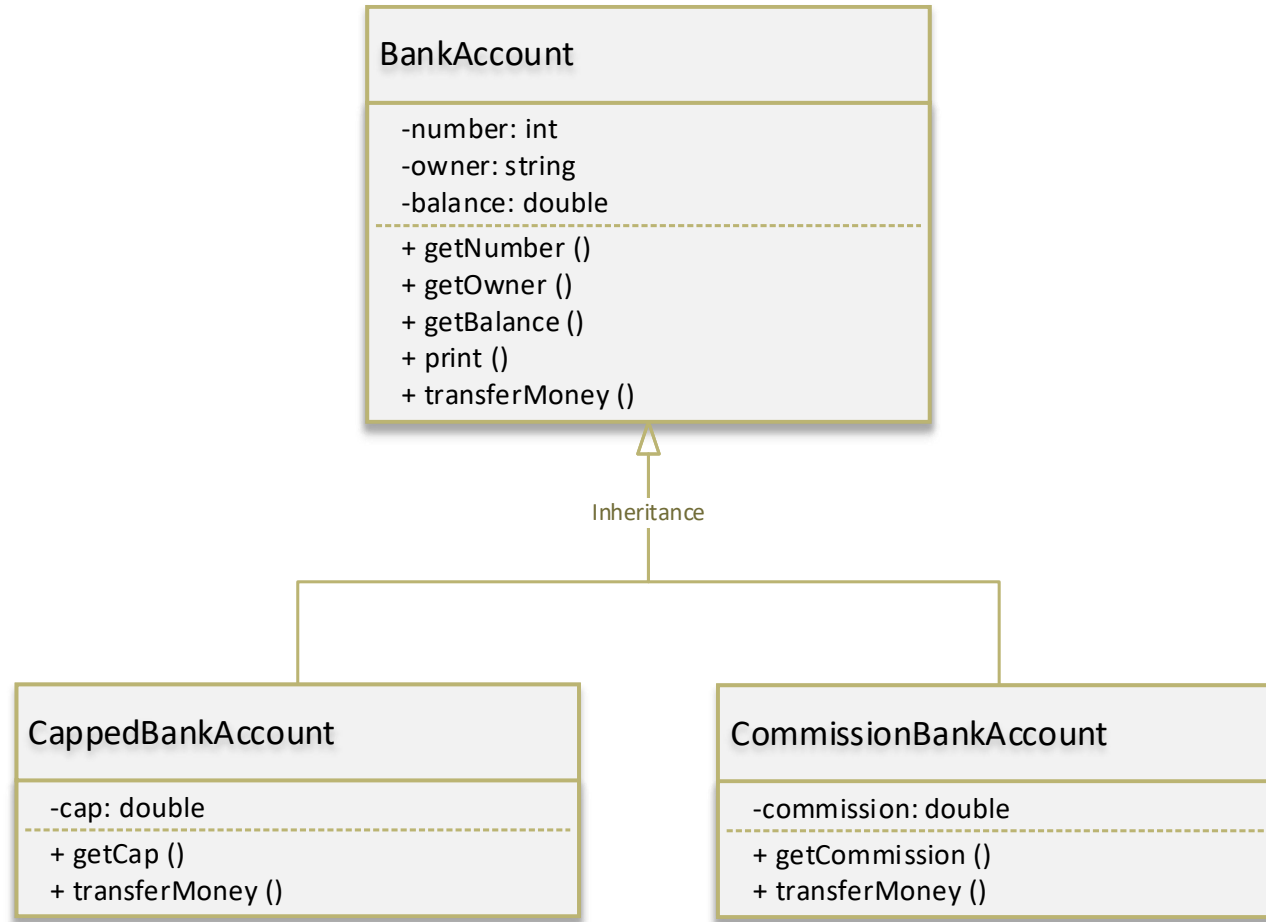
Extending BankAccount

- During lecture 6, we implemented a class BankAccount to handle bank accounts
- We need to extend BankAccount to support two new kinds of bank account:
 - CappedBankAccount
 - Transfers have a cap
 - When money is transferred, if the amount being transferred exceeds the cap, the transfer fails
 - CommissionBankAccount
 - Transfers are subject to account-specific commission
 - When money is transferred, the sender is charged the commission

Inheritance

- A subtyping relation says: every instance of a subtype is (also) an instance of a supertype
 - In arithmetic, every integer number is a real number
 - In geometry, every square is a rectangle
 - In a program for managing bank accounts, every CappedBankAccount is a BankAccount
- The supertype supports general operations; the subtype may have specialized operations
- When is this useful?
 - Bottom-up perspective (generalization)
 - We have many bank account classes with shared functionalities. let's group them together
 - Top-down perspective (specialization)
 - A BankAccount class distinguishes different kinds of bank accounts. let's make separate classes
- Advantages: modularity, clarity, maintainability

Class diagrams



Derived classes

```
1 class BaseClass {  
2     private:  
3         int var;  
4         void increment(int);  
5     public:  
6         BaseClass(int);  
7         int getVar();  
8         void print();  
9 }  
10  
11 class DerivedClass : public BaseClass {  
12     private:  
13         string text;  
14     public:  
15         DerivedClass(string);  
16         string getText();  
17         void print();  
18 }
```

- When defining a class, we can specify that the class is derived from another class
 - Public members are inherited
 - Private methods are not
 - Private attributes are inherited, but not accessible by the derived class

Derived classes

```
1 class BaseClass {  
2     private:  
3         int var;  
4         void increment(int);  
5     public:  
6         BaseClass(int);  
7         int getVar();  
8         void print();  
9 }  
10  
11 class DerivedClass : public BaseClass {  
12     private:  
13         string text;  
14     public:  
15         DerivedClass(string);  
16         string getText();  
17         void print();  
18 }
```

- When defining a class, we can specify that the class is derived from another class
 - Public members are inherited
 - Private methods are not
 - Private attributes are inherited, but not accessible by the derived class
- The derived class can have its own members, in addition to the inherited ones

Derived classes

```
1 BaseClass::BaseClass(int i) {  
2     this->var = i;  
3 }  
4  
5 void BaseClass::print(){  
6     cout << "the value of var is " << this->var << endl;  
7 }  
8  
9 DerivedClass::DerivedClass(string s) : BaseClass(10) {  
10     this->text = s;  
11 }  
12  
13 void DerivedClass::print(){  
14     cout << "the value of text is " << this->text << ", and ";  
15     BaseClass::print();  
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
 - Private attributes of the base class cannot be accessed by the derived class
 - How to initialize them?

Derived classes

```
1 BaseClass::BaseClass(int i) {  
2     this->var = i;  
3 }  
4  
5 void BaseClass::print(){  
6     cout << "the value of var is " << this->var << endl;  
7 }  
8  
9 DerivedClass::DerivedClass(string s) : BaseClass(10) {  
10     this->text = s;  
11 }  
12  
13 void DerivedClass::print(){  
14     cout << "the value of text is " << this->text << ", and ";  
15     BaseClass::print();  
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
 - Private attributes of the base class cannot be accessed by the derived class
 - How to initialize them?
 - By calling the constructor of the base class

Derived classes

```
1 BaseClass::BaseClass(int i) {  
2     this->var = i;  
3 }  
4  
5 void BaseClass::print(){  
6     cout << "the value of var is " << this->var << endl;  
7 }  
8  
9 DerivedClass::DerivedClass(string s) : BaseClass(10) {  
10     this->text = s;  
11 }  
12  
13 void DerivedClass::print(){  
14     cout << "the value of text is " << this->text << ", and ";  
15     BaseClass::print();  
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
 - Private attributes of the base class cannot be accessed by the derived class
 - How to initialize them?
 - By calling the constructor of the base class
- We can override (i.e., redefine) inherited methods, so to specialize their code
 - The overridden method must be specified in the derived class definition

Derived classes

```
1 BaseClass::BaseClass(int i) {  
2     this->var = i;  
3 }  
4  
5 void BaseClass::print(){  
6     cout << "the value of var is " << this->var << endl;  
7 }  
8  
9 DerivedClass::DerivedClass(string s) : BaseClass(10) {  
10     this->text = s;  
11 }  
12  
13 void DerivedClass::print(){  
14     cout << "the value of text is " << this->text << ", and ";  
15     BaseClass::print();  
16 }
```

- Constructors are not inherited, and need to be implemented for derived classes
 - Private attributes of the base class cannot be accessed by the derived class
 - How to initialize them?
 - By calling the constructor of the base class
- We can override (i.e., redefine) inherited methods, so to specialize their code
 - The overridden method must be specified in the derived class definition
 - We can reuse the method in the base class by invoking it in the overridden method

Encapsulation and inheritance

```
1 class BaseClass {  
2     public:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 void DerivedClass::print(){  
13     cout << "the contents are the following: ";  
14     cout << "text is " << this->text <<  
15     cout << ", and var is " << this->var << endl;  
16 }
```

- Public members are accessible from outside the class
 - They break the encapsulation principle
- Private members are not accessible from derived classes
 - We rely on public methods of the base class to interact with them
 - In many cases, this is not enough
- Solution: declare members that must be accessible from derived class as protected:
 - Inherited by derived classes
 - Accessible by derived classes
 - Non accessible from outside the class

Encapsulation and inheritance

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : public BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }
```

- What happens to the interface of a derived class?

Encapsulation and inheritance

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : public BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }
```

- What happens to the interface of a derived class?
- It depends on how it extends the base class:
 - With the public modifier:
 - Public members will remain public
 - Protected members will remain protected

Encapsulation and inheritance

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : protected BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }
```

- What happens to the interface of a derived class?
- It depends on how it extends the base class:
 - With the public modifier:
 - Public members will remain public
 - Protected members will remain protected
 - With the protected modifier:
 - Public members will become protected
 - Protected members will remain protected

Encapsulation and inheritance

```
1 class BaseClass {
2     private:
3         void increment(int);
4     protected:
5         int var;
6     public:
7         BaseClass(int);
8         int getVar();
9         void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
```

- What happens to the interface of a derived class?
- It depends on how it extends the base class:
 - With the public modifier:
 - Public members will remain public
 - Protected members will remain protected
 - With the protected modifier:
 - Public members will become protected
 - Protected members will remain protected
 - With the private modifier:
 - Public members will become private
 - Protected members will become private

Encapsulation and inheritance

Derived class	Public members	Protected members	Private members
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Private	Private	Private

Overridden methods dispatch

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : private BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }  
20  
21 DerivedClass *d = new DerivedClass();  
22 BaseClass *b = d;  
23 d->print();  
24 b->print();
```

- Which method is invoked by d-> print()?

Overridden methods dispatch

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : private BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }  
20  
21 DerivedClass *d = new DerivedClass();  
22 BaseClass *b = d;  
23 d->print();  
24 b->print();
```

- Which method is invoked by d-> print()?
 - DerivedClass::print()

Overridden methods dispatch

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : private BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }  
20  
21 DerivedClass *d = new DerivedClass();  
22 BaseClass *b = d;  
23 d->print();  
24 b->print();
```

- Which method is invoked by d-> print()?
 - DerivedClass::print()
- Which method is invoked by b-> print()?

Overridden methods dispatch

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : private BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }  
20  
21 DerivedClass *d = new DerivedClass();  
22 BaseClass *b = d;  
23 d->print();  
24 b->print();
```

- Which method is invoked by d-> print()?
 - DerivedClass::print()
- Which method is invoked by b-> print()?
 - BaseClass::print()
 - Why?
 - Because C++ uses static method dispatch:
 - The method is determined by the data type, and not by the object

Overridden methods dispatch

```
1 class BaseClass {  
2     private:  
3         void increment(int);  
4     protected:  
5         int var;  
6     public:  
7         BaseClass(int);  
8         int getVar();  
9         void print();  
10 }  
11  
12 class DerivedClass : private BaseClass {  
13     private:  
14         string text;  
15     public:  
16         DerivedClass(string);  
17         string getText();  
18         void print();  
19 }  
20  
21 DerivedClass *d = new DerivedClass();  
22 BaseClass *b = d;  
23 d->print();  
24 b->print();
```

- Which method is invoked by d-> print()?
 - DerivedClass::print()
- Which method is invoked by b-> print()?
 - BaseClass::print()
 - Why?
 - Because C++ uses static method dispatch:
 - The method is determined by the data type, and not by the object
- Can we force DerivedClass::print() to be invoked instead?

Overridden methods dispatch

```
1 class BaseClass {
2     private:
3         void increment(int);
4     protected:
5         int var;
6     public:
7         BaseClass(int);
8         int getVar();
9         virtual void print();
10 }
11
12 class DerivedClass : private BaseClass {
13     private:
14         string text;
15     public:
16         DerivedClass(string);
17         string getText();
18         void print();
19 }
20
21 DerivedClass *d = new DerivedClass();
22 BaseClass *b = d;
23 d->print();
24 b->print();
```

- Which method is invoked by `d-> print()`?
 - `DerivedClass::print()`
- Which method is invoked by `b-> print()`?
 - `BaseClass::print()`
 - Why?
 - Because C++ uses static method dispatch:
 - The method is determined by the data type, and not by the object
 - Can we force `DerivedClass::print()` to be invoked instead?
 - Yes, with dynamic method dispatch
 - We have to mark `print()` as virtual in `BaseClass`
 - Slower but usually more intuitive

Abstract classes

```
1 class BaseClass {
2     private:
3         void increment(int);
4     protected:
5         int var;
6     public:
7         BaseClass(int);
8         virtual void print() = 0;
9 }
10
11 class DerivedClass : private BaseClass {
12     private:
13         string text;
14     public:
15         DerivedClass(string);
16         string getText();
17         void print();
18 }
19
20 void DerivedClass::print(){
21     cout << "the contents are the following: ";
22     cout << "text is " << this->text <<
23     cout << ", and var is " << this->var << endl;
24 }
```

- A class is abstract if it contains at least one “pure virtual” method:
 - Marked by “= 0”
 - Defined but not implemented

Abstract classes

```
1 class BaseClass {
2     private:
3         void increment(int);
4     protected:
5         int var;
6     public:
7         BaseClass(int);
8         virtual void print() = 0;
9 }
10
11 class DerivedClass : private BaseClass {
12     private:
13         string text;
14     public:
15         DerivedClass(string);
16         string getText();
17         void print();
18 }
19
20 void DerivedClass::print(){
21     cout << "the contents are the following: ";
22     cout << "text is " << this->text <<
23     cout << ", and var is " << this->var << endl;
24 }
```

- A class is abstract if it contains at least one “pure virtual” method:
 - Marked by “= 0”
 - Defined but not implemented
- Abstract classes cannot be instantiated, only derived
- A class derived from an abstract class can be instantiated only if it implements all the pure virtual methods of the base class

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 8
 - Ask questions if something is unclear (including previous assignments)

DTU



02393 Programming in C++

Module 10: Recursive Programming

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Recursion
- Complexity
- Lab

A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`

Recursion

- Recursive programming is a solution technique that solves large problems by reducing them to smaller problems of the same form
 - It is crucial that both the large and smaller problems have the same form!
 - So we can use the same technique (and the same code!) to solve both
- Why might recursion seem... weird?
 - Recursion requires a form of mathematical inductive reasoning
 - Recursion is more abstract than programming concepts having a mechanical intuition
 - loop: repeat an action several times
 - if-then-else: check a condition, make a decision
 - recursion: ???

Recursion

```
1 unsigned int factorial = 1;
2   for (unsigned int i = n; i > 0; i --) {
3       factorial = factorial * i;
4   }
5
6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }
```

- You may have already seen recursion in mathematical definitions. E.g., the factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot ((n-1)!) & \text{otherwise} \end{cases}$$

- We can implement the factorial in C++ with an iterative approach

Recursion

```
1 unsigned int factorial = 1;
2   for (unsigned int i = n; i > 0; i --) {
3       factorial = factorial * i;
4   }
5
6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }
```

- You may have already seen recursion in mathematical definitions. E.g., the factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot ((n-1)!) & \text{otherwise} \end{cases}$$

- We can implement the factorial in C++ with an iterative approach
- ...But we can also use recursion.

Recursion

```
1 unsigned int factorial = 1;
2 for (unsigned int i = n; i > 0; i --) {
3     factorial = factorial * i;
4 }
5
6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }
```

- You may have already seen recursion in mathematical definitions. E.g., the factorial:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot ((n-1)!) & \text{otherwise} \end{cases}$$

- We can implement the factorial in C++ with an iterative approach
- ...But we can also use recursion.
 - Fact directly returns a value only when $n = 0$ (base case)
 - Otherwise, Fact invokes itself passing $n-1$
 - Eventually, the base case is reached
- Note that the implementation is closer to the mathematical definition.

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Since $n=3$ is not equal to 0, Fact(3) will invoke Fact(2)
 - The execution of Fact(3) is paused until Fact(2) returns

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Since $n=2$ is not equal to 0, Fact(2) will invoke Fact(1)
 - The execution of Fact(2) is paused until Fact(1) returns

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Since $n=1$ is not equal to 0, Fact(1) will invoke Fact(0)
 - The execution of Fact(1) is paused until Fact(0) returns

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??
Fact(1)	0x1040	n = 1
	0x1044	r = ??

Executing Fact(3)

```

6 unsigned int Fact ( unsigned int n) {
7     if (n == 0) {
8         return 1;
9     } else {
10        return n * Fact (n -1);
11    }
12 }

```

- Since $n = 0$ is the base case, Fact(0) will return 1

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??
Fact(1)	0x1040	n = 1
	0x1044	r = ??
Fact(0)	0x1060	n = 0
	0x1064	r = 1

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Fact(1) will then resume execution, multiplying the output of Fact(0) with n and returning the result

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = ??
Fact(1)	0x1040	n = 1
	0x1044	r = 1

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Fact(2) will then resume execution, multiplying the output of Fact(1) with n and returning the result

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = ??
Fact(2)	0x1020	n = 2
	0x1024	r = 2

Executing Fact(3)

```
6 unsigned int Fact ( unsigned int n) {  
7     if (n == 0) {  
8         return 1;  
9     } else {  
10        return n * Fact (n -1);  
11    }  
12 }
```

- Fact(3) will then resume execution, multiplying the output of Fact(2) with n and returning the result

Function stack	Address	Contents
Fact(3)	0x1000	n = 3
	0x1004	r = 6

Recursion

- When using recursion, we must ensure that:
 1. there are one or more base cases
 - i.e., smallest problems for which we return a solution, without recursion
 2. every recursion step reduces to a smaller problem
 3. every sequence of recursion steps eventually reaches one of the base cases
- In the Factorial example: the base case is $n = 0$, we recursively call Fact on smaller and smaller numbers, so we eventually reach the base case $n = 0$
- If any of the 3 conditions above is not satisfied, recursion may not work properly!
 - Risk of non-termination or crashes (stack overflow)
- When writing a recursive function, we need to assume that each recursive call with a smaller problem computes the correct solution
 - Example: to write Fact(n), we need to assume that Fact($n-1$) is correct
- Assuming that a recursive call works correctly is called the “recursive leap of faith”

Recursion: rules of thumb

- Identify the base cases (i.e., the “smallest” cases whose solution does not need recursion)
- Solve the base cases
- Address the recursive cases (i.e., the non-base cases that need recursive calls)
 - Ensure that the arguments to the recursive calls are “smaller” than the original arguments
 - Ensure that recursive calls eventually reach one of the base cases

Complexity

- We are often interested in estimating the resources needed by an algorithm:
 - time: number of operations
 - space: amount of memory/disk
- Such resources usually depend on the size of the algorithm inputs (denoted N)
- Complexity of a problem: given a concrete problem (e.g., sorting a list of numbers) what time/space resources are needed by the best solution algorithm?
- Notes:
 - Some problems are not computable! (i.e., no algorithm exists)
 - Sometimes we have a trade-off between time and space
 - For many problems, the precise complexity is not known:
 - we may know some solution algorithms, but we don't know whether a better one exists
 - we can give a lower bound to the complexity

Complexity

- We are often interested in the worst-case time / space requirements of an algorithm, given an input size N . In such cases, we use Big-O notation. E.g.:
 - $2N^2 + 17N + 53$ operations $\rightarrow O(N^2)$ time complexity
- We only consider dominant terms. This is because, as N grows,
 - larger exponents have more impact
 - constant factors and minor terms tend to become irrelevant
- For example:
 1. A good algorithm, time: $3000N \rightarrow O(N)$ time complexity
 2. A bad algorithm, time: $2N^2 \rightarrow O(N^2)$ time complexity
- Above some N , the algorithm (1) performs better (less operations) than (2)

More examples of complexity (see lecture code)

- Efficient search: binary search
 - Naive search (linear search) of an element in a set takes $O(n)$
 - Binary search is a divide-and-conquer $O(\log n)$ solution
- Efficient sorting: merge sort
 - The recursion paradigm directly triggers an efficient solution!
 - Naive bubble sort: $O(n^2)$ for array of size n
 - Merge sort: $O(n \log n)$ (theoretical optimum)
- Efficient exponentiation in cryptography ($a^n \bmod p$)
 - Naive exponentiation: $O(n)$
 - Efficient exponentiation: $O(\log n)$
 - Efficient solution is hard to program without recursion!

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 9
 - Ask questions if something is unclear (including previous assignments)

DTU



02393 Programming in C++

Module 11: Linked Lists

© Giovanni Meroni

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Single-linked lists
- Doubly-linked lists
- Lab

A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`
- Recursive programming
 - Recursive functions

Recursion: rules of thumb

- Identify the base cases (i.e., the “smallest” cases whose solution does not need recursion)
- Solve the base cases
- Address the recursive cases (i.e., the non-base cases that need recursive calls)
 - Ensure that the arguments to the recursive calls are “smaller” than the original arguments
 - Ensure that recursive calls eventually reach one of the base cases

Recursive data types

- Many mathematical entities can be recursively defined:
 - A natural number is either 0, or the successor of a natural number
 - A set can be either empty, or a singleton, or the union of two sets
- Similarly, many data types can be recursively defined:
 - A list can be either empty, or one item, or the concatenation of two lists
 - A tree can be either empty, or one leaf node, or an internal node with two sub-trees

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 class Node {  
7 public:  
8     int content;  
9     Node *next;  
10 };
```

- A Node instance has some content and points to the next Node in the list
 - Nodes can be implemented as structures or classes
- A linked list is usually dynamically allocated, and nodes may be added or removed
- Therefore, a linked list is accessed through a pointer to a Node (i.e., Node*)
 - The pointer nullptr represents an empty list
 - Otherwise, it is a pointer to the first node of the list
- Important: do not forget to initialize your pointers!

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 Node* findNodeIterative(Node *head, int value) {  
7     while (head != nullptr){  
8         if (head->content == value) return head;  
9         head = head->next;  
10    }  
11    return nullptr;  
12 }  
13  
14 Node* findNodeRecursive(Node *currNode, int value) {  
15     if (currNode == nullptr){  
16         return nullptr;  
17     } else if (currNode->content == value){  
18         return currNode;  
19     } else {  
20         return findNodeRecursive(currNode->next, value);  
21     }  
22 }
```

- To find nodes in a list, one has to:
 - Check if the end of the list has been reached

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 Node* findNodeIterative(Node *head, int value) {  
7     while (head != nullptr){  
8         if (head->content == value) return head;  
9         head = head->next;  
10    }  
11    return nullptr;  
12 }  
13  
14 Node* findNodeRecursive(Node *currNode, int value) {  
15     if (currNode == nullptr){  
16         return nullptr;  
17     } else if (currNode->content == value){  
18         return currNode;  
19     } else {  
20         return findNodeRecursive(currNode->next, value);  
21     }  
22 }
```

- To find nodes in a list, one has to:
 - Check if the end of the list has been reached
 - Check if the current node contains the specified value

Single-linked lists

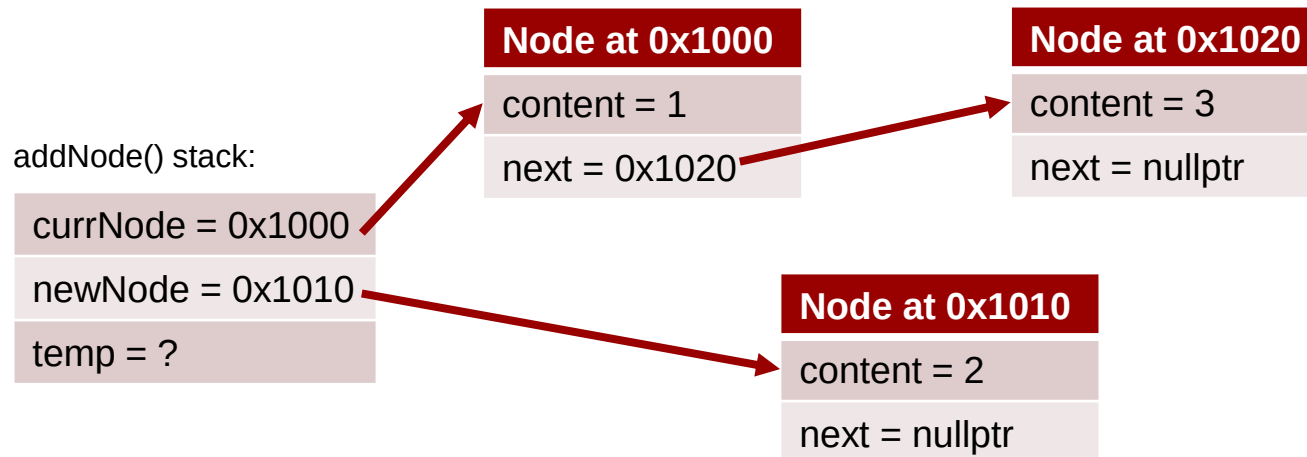
```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 Node* findNodeIterative(Node *head, int value) {  
7     while (head != nullptr){  
8         if (head->content == value) return head;  
9         head = head->next;  
10    }  
11    return nullptr;  
12 }  
13  
14 Node* findNodeRecursive(Node *currNode, int value) {  
15     if (currNode == nullptr){  
16         return nullptr;  
17     } else if (currNode->content == value){  
18         return currNode;  
19     } else {  
20         return findNodeRecursive(currNode->next, value);  
21     }  
22 }
```

- To find nodes in a list, one has to:
 - Check if the end of the list has been reached
 - Check if the current node contains the specified value
 - Move to the next node
- Once in the next node, previous nodes can no longer be reached

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

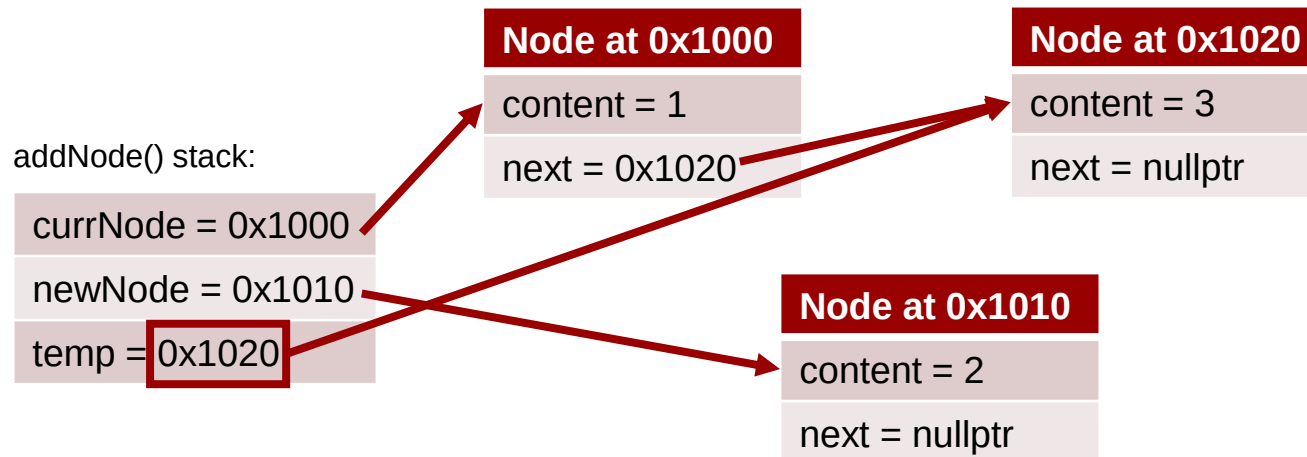
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

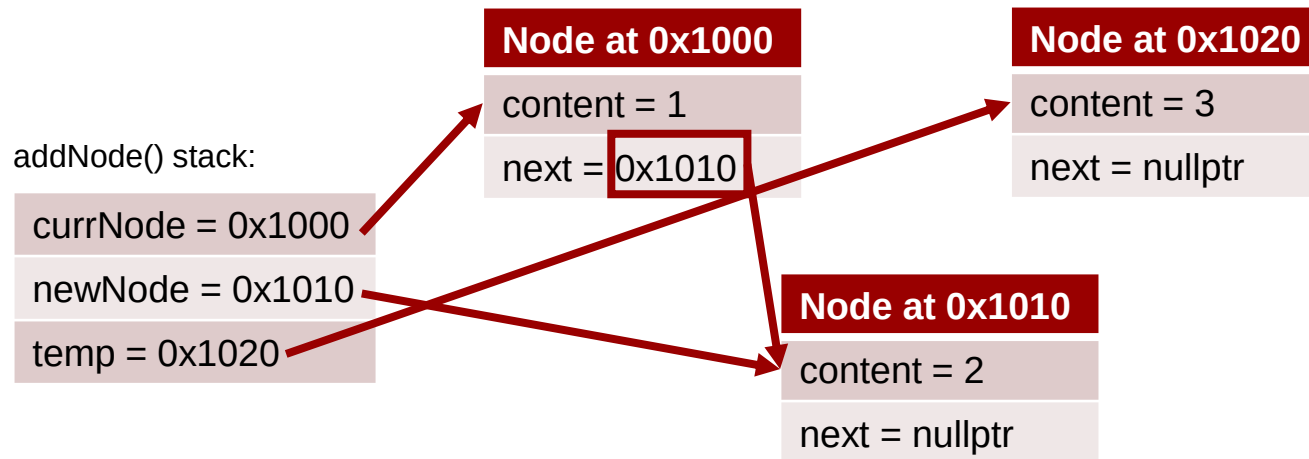
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node



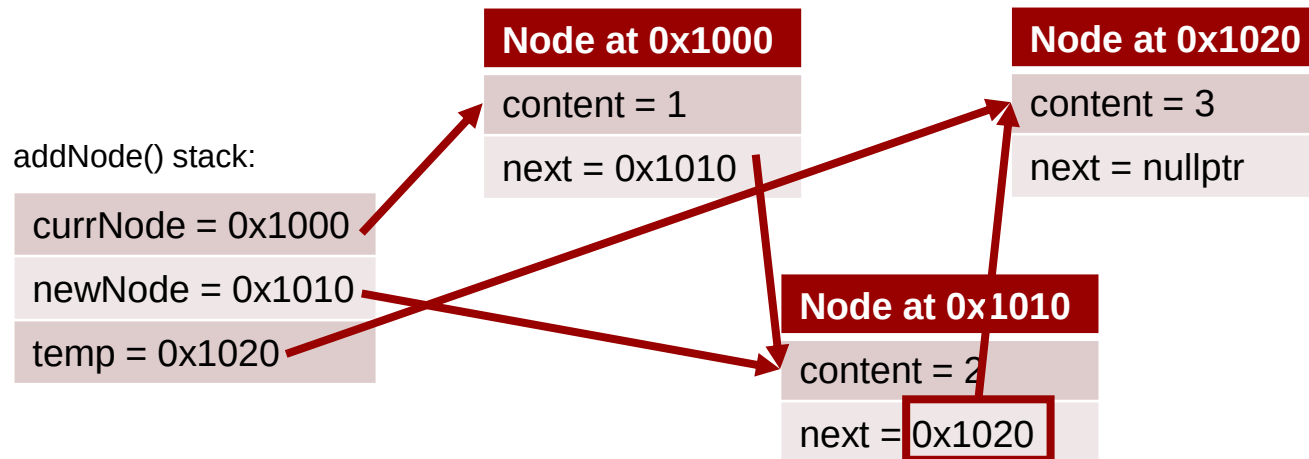
Single-linked lists

```

1 struct Node {
2     int content;
3     Node *next;
4 }
5
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10 }

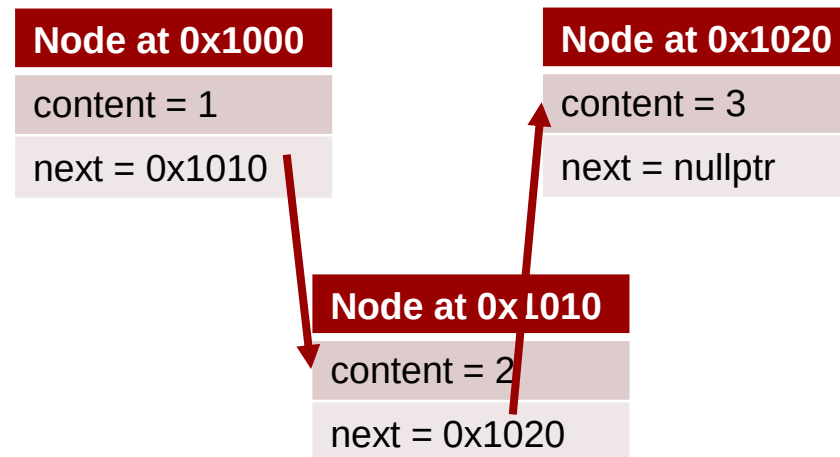
```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
- Assign the address of the old next node to the pointer to the next node in the new node



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```



- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
- Assign the address of the old next node to the pointer to the next node in the new node

Single-linked lists

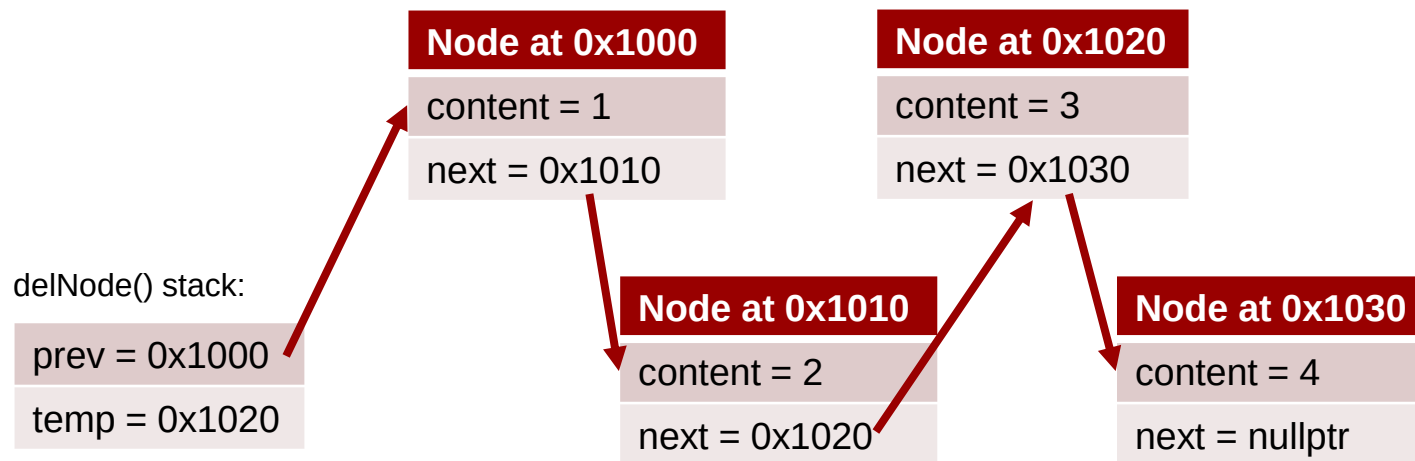
```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10 }
```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the old next node to the pointer to the next node in the new node
- Special cases (not in this code):
 - The list is empty
 - We need to add a new node in front of the list

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

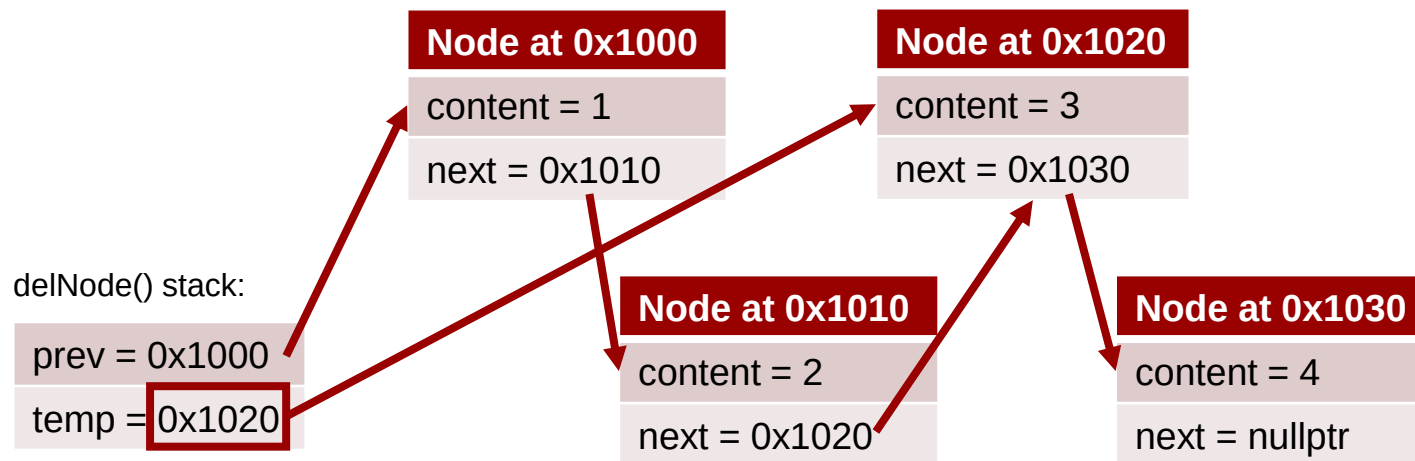
- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted



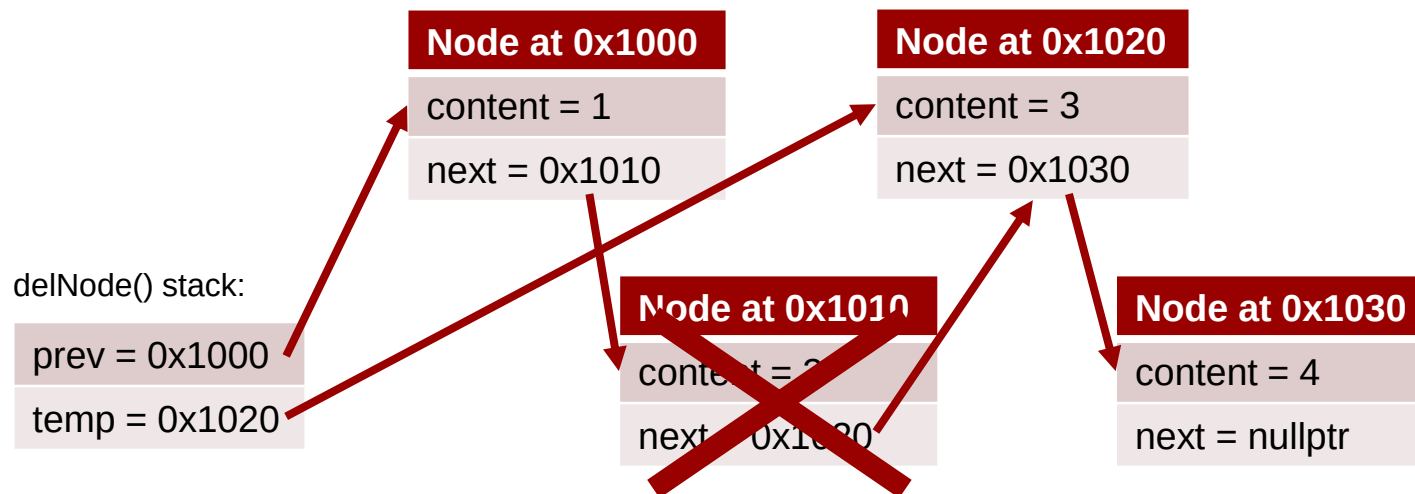
Single-linked lists

```

1 struct Node {
2     int content;
3     Node *next;
4 }
5
6 void delNode(Node *prev) {
7     Node temp = prev->next->next;
8     delete prev->next;
9     prev->next = temp;
10 }

```

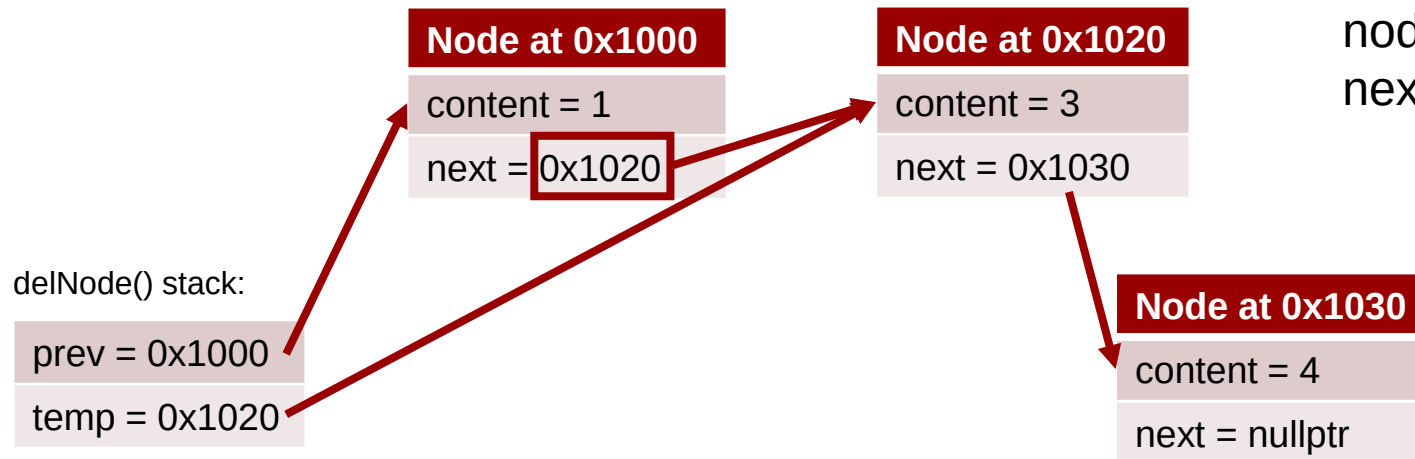
- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor



Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

Node at 0x1000

content = 1

next = 0x1020

Node at 0x1020

content = 3

next = 0x1030

Node at 0x1030

content = 4

next = nullptr

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor

Single-linked lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4 }  
5  
6 void delNode(Node *prev) {  
7     Node temp = prev->next->next;  
8     delete prev->next;  
9     prev->next = temp;  
10 }
```

- When deleting a node from a list, one needs to:
 - Locate the predecessor of that node
 - Save the address of the successor of the node to be deleted
 - Deallocate the memory used by the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor
- Special cases (not in this code):
 - The node to be deleted is the first
 - The list has only one node
 - The list is empty

Single-linked lists

- In previous lectures we saw an example of how to implement a vector-like class based on arrays
- Today we are going to see how to implement:
 - A vector-like class based on linked lists
- In today's lecture code on DTU Learn you will also find:
 - A set-like class based on linked lists
- Note: the C++ STL provides linked lists. See:
<http://en.cppreference.com/w/cpp/container/list>

Doubly-linked lists

- Some annoying tasks with single-linked lists:
 - Delete a pointed element (how do we update the previous element?)
 - Concatenate two lists (time complexity $O(N)$, as we need to reach the end of the first list)
- One possible solution: doubly-linked lists

Doubly-Linked Lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4     Node *prev  
5 }  
6  
7 class Node {  
8 public:  
9     int content;  
10    Node *next;  
11    Node *prev;  
12 };
```

- Node instance has some content and points to the next and previous Nodes in the list
 - The first element in the list will have prev set to nullptr
 - The last element in the list will have next set to nullptr

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp != nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = ?

Node at 0x1000

content = 1

next = 0x1020

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1000

Node at 0x1010

content = 2

next = nullptr

prev = nullptr

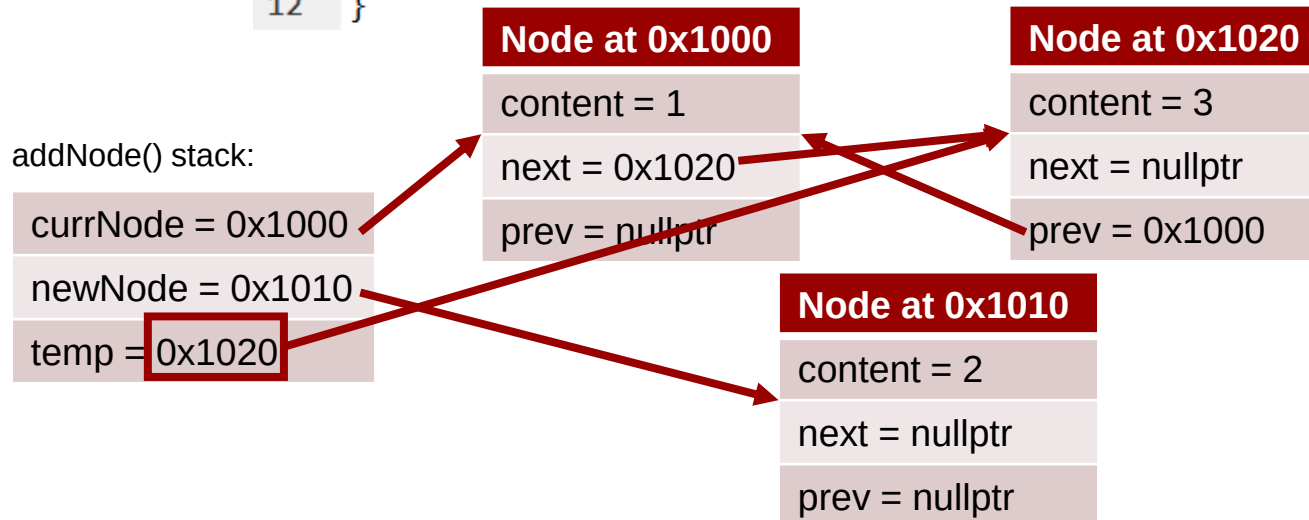
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node



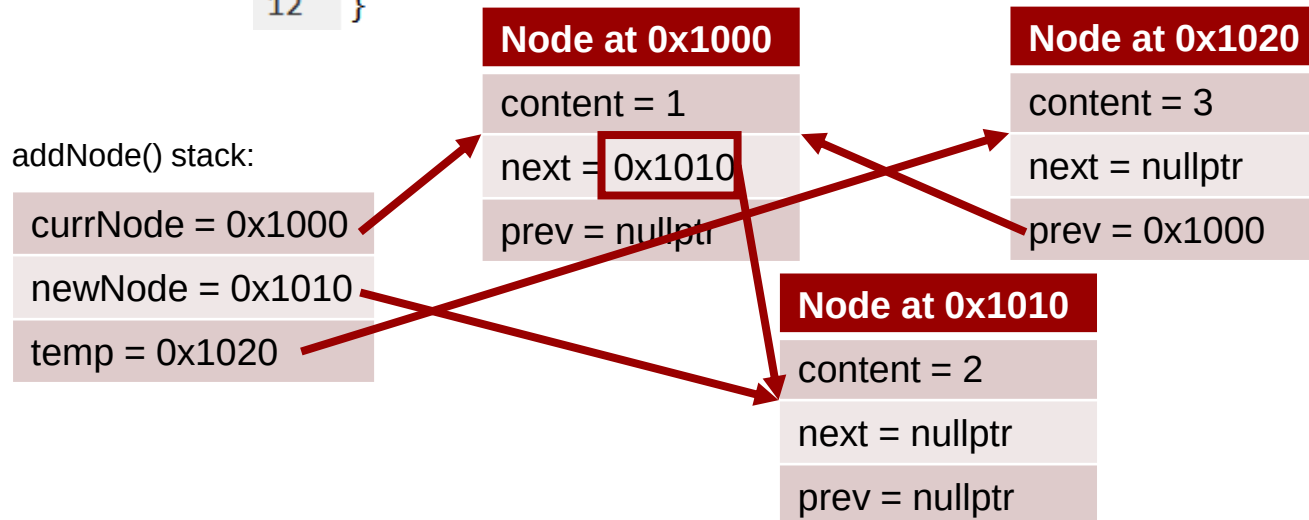
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node



Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = 0x1020

Node at 0x1000

content = 1

next = 0x1010

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1000

Node at 0x1010

content = 2

next = 0x1020

prev = nullptr

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = 0x1020

Node at 0x1000

content = 1

next = 0x1010

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1000

Node at 0x1010

content = 2

next = 0x1020

prev = 0x1000

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```

addNode() stack:

currNode = 0x1000

newNode = 0x1010

temp = 0x1020

Node at 0x1000

content = 1

next = 0x1010

prev = nullptr

Node at 0x1020

content = 3

next = nullptr

prev = 0x1010

Node at 0x1010

content = 2

next = 0x1020

prev = 0x1000

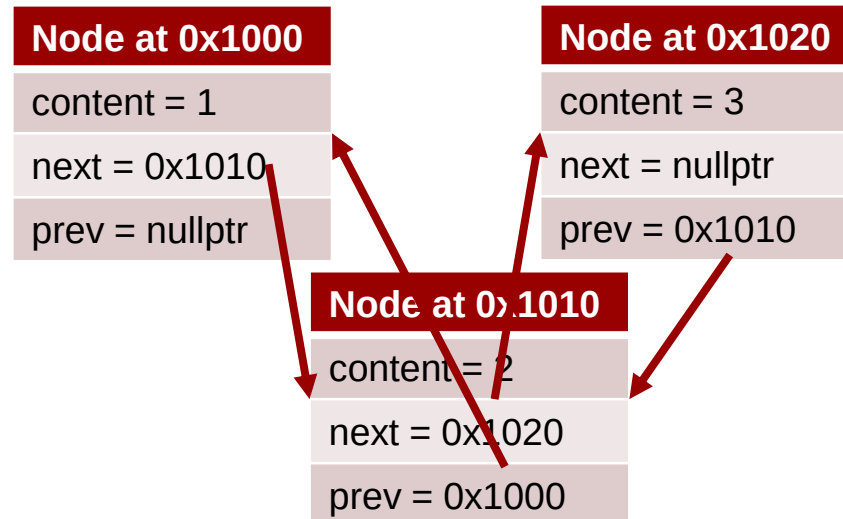
- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**
 - **Assign the address of the new node to the pointer of the previous node in the old next node (if it exists)**

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void addNode(Node *currNode, Node *newNode){
7     Node *temp = currNode->next;
8     currNode->next = newNode;
9     newNode->next = temp;
10    newNode->prev = currNode;
11    if(temp!=nullptr) temp->prev = newNode;
12 }

```



- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**
 - **Assign the address of the new node to the pointer of the previous node in the old next node (if it exists)**

Doubly-Linked Lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4     Node *prev  
5 }  
6 void addNode(Node *currNode, Node *newNode){  
7     Node *temp = currNode->next;  
8     currNode->next = newNode;  
9     newNode->next = temp;  
10    newNode->prev = currNode;  
11    if(temp!=nullptr) temp->prev = newNode;  
12 }
```

- Special cases (not in this code):
 - The list is empty
 - We need to add a new node in front of the list

- When adding a new node to a list, one needs to:
 - Locate the node after which the new node will be inserted
 - Save the address of the next node of the current node
 - Assign the address of the new node to the pointer of the next node of the current node
 - Assign the address of the new node to the pointer of the next node in the current node
 - **Assign the address of the current node to the pointer of the previous node in the new node**
 - **Assign the address of the new node to the pointer of the previous node in the old next node (if it exists)**

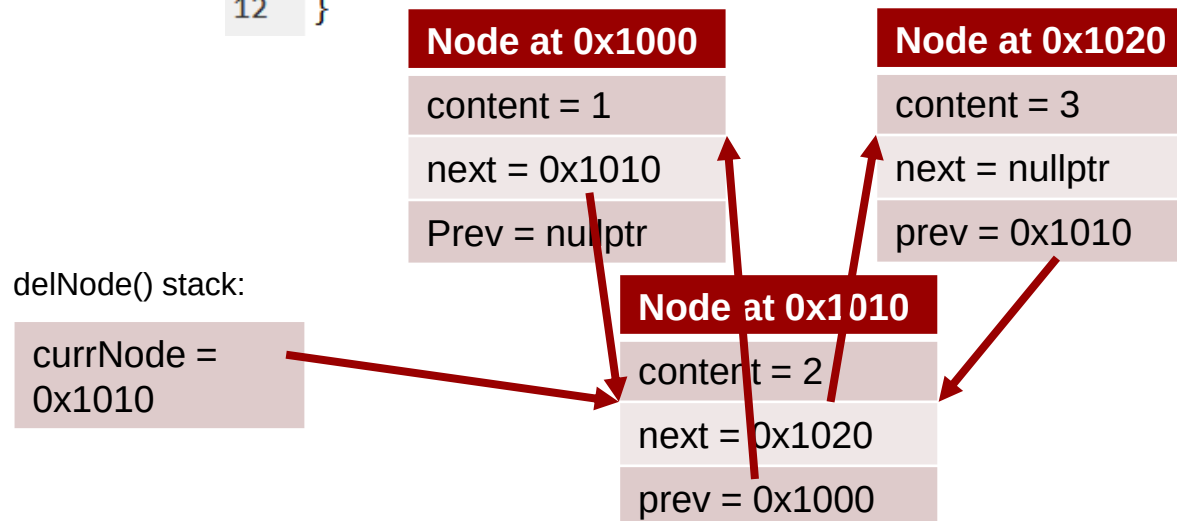
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11    delete currNode;
12 }

```

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted



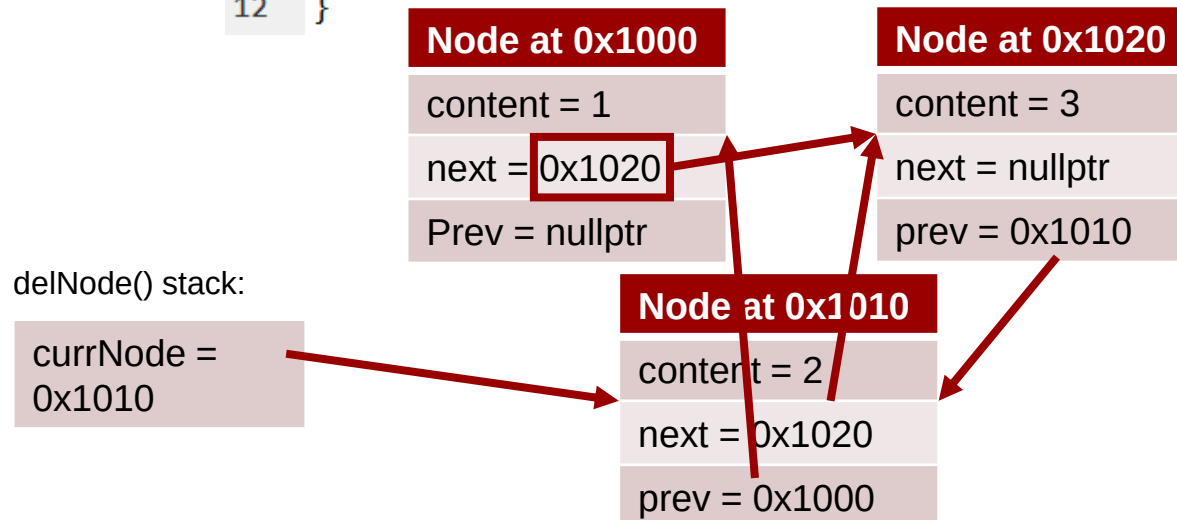
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11    delete currNode;
12 }

```

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)



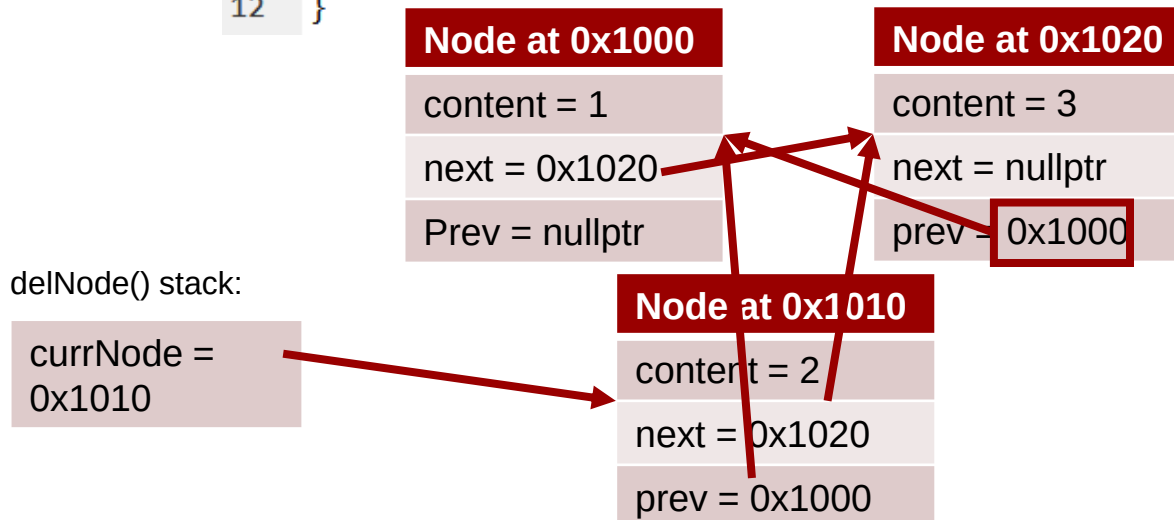
Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11    delete currNode;
12 }

```

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)

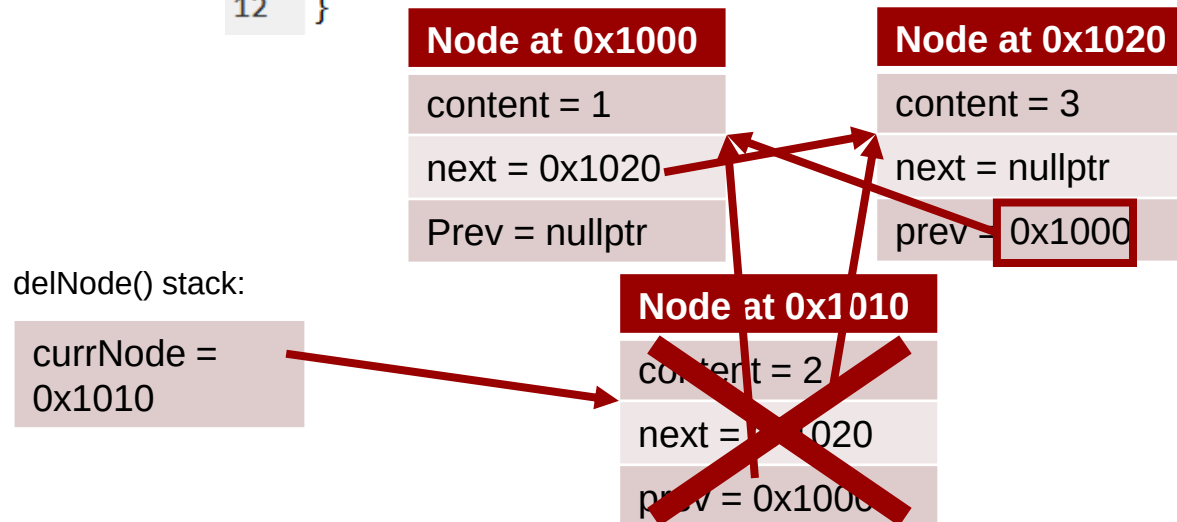


Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11     delete currNode;
12 }

```



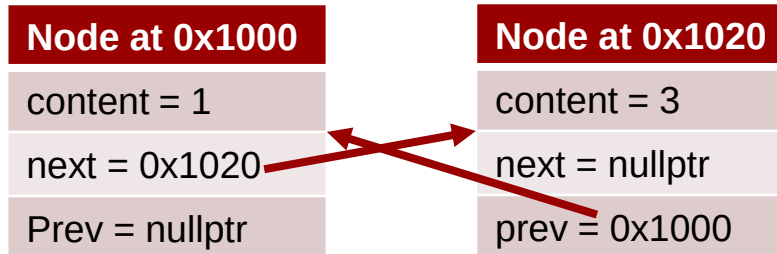
- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)
 - Deallocate the memory used by the node to be deleted

Doubly-Linked Lists

```

1 struct Node {
2     int content;
3     Node *next;
4     Node *prev
5 }
6 void delNode(Node *currNode) {
7     if(currNode->prev != nullptr)
8         currNode->prev->next = currNode->next;
9     if(currNode->next != nullptr)
10        currNode->next->prev = currNode->prev;
11     delete currNode;
12 }

```



- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)
 - Deallocate the memory used by the node to be deleted

Doubly-Linked Lists

```
1 struct Node {  
2     int content;  
3     Node *next;  
4     Node *prev  
5 }  
6 void delNode(Node *currNode) {  
7     if(currNode->prev != nullptr)  
8         currNode->prev->next = currNode->next;  
9     if(currNode->next != nullptr)  
10        currNode->next->prev = currNode->prev;  
11     delete currNode;  
12 }
```

- Special cases (not in this code):
 - The node to be deleted is the first
 - The node to be deleted is the last
 - The list has only one node
 - The list is empty

- When deleting a node from a list, one needs to:
 - Locate the node to be deleted
 - Assign the address of the successor of node to be deleted to the pointer to the next node of the predecessor (if any)
 - Assign the address of the predecessor of the node to be deleted to the pointer to the previous node of the successor (if any)
 - Deallocate the memory used by the node to be deleted

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 10
 - Ask questions if something is unclear (including previous assignments)

DTU



02393 Programming in C++

Module 12: Trees

© **Giovanni Meroni**

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	Inheritance	19.1 - 19.3
9	05/11	LAB DAY	N/A
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

- Recap
- Trees
- Trees in C++
- Using trees to represent expressions
- Lab

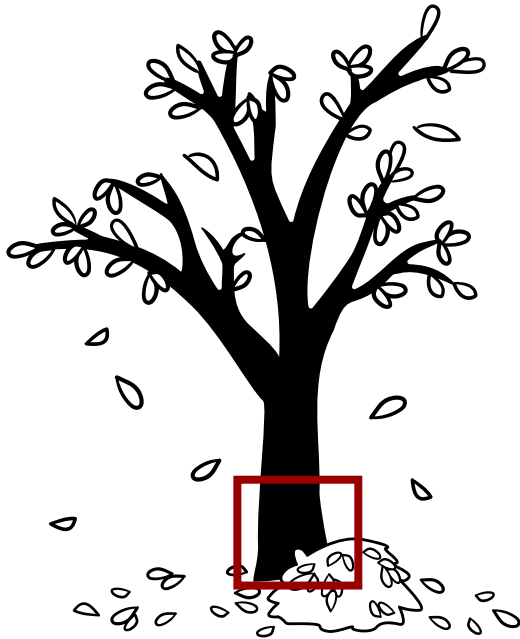
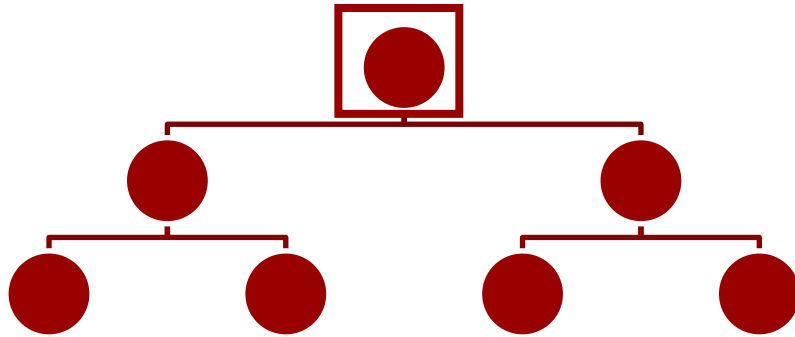
A recap from the previous lectures

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions and methods (including constructors, destructors, operators), templates
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`, `vector`, `ifstream`, `ofstream`, `class`, `this`
- Expressions
 - some numeric and boolean operators and math functions, conditional expressions
- Statements
 - `if`, `while`, `for`, `switch`
- Recursive programming
 - Recursive functions, linked lists

Recursive programming

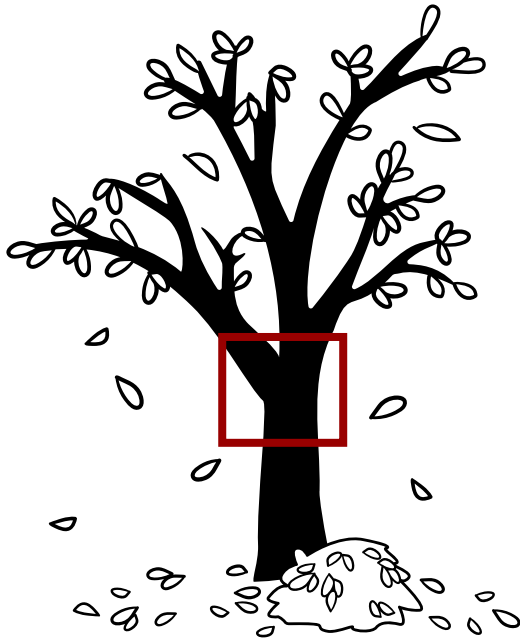
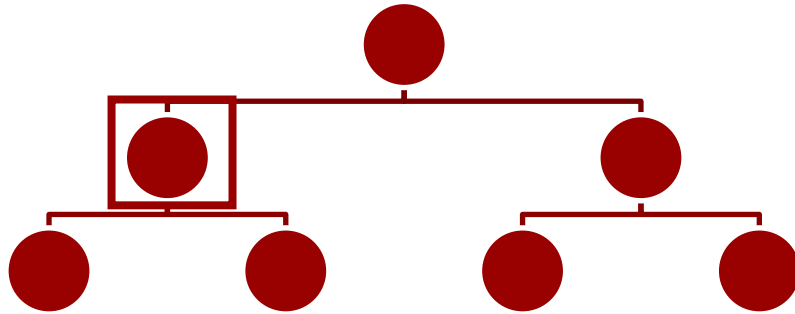
- Recursive programming is useful when dealing with
 - Recursively-defined problems (e.g. computing the factorial of a number)
 - Recursive data structures
- Examples of recursive data structures:
 - Linked lists (last lecture): singly-linked and doubly-linked
 - Trees (today!)
 - Sets, multi-sets
- Always keep in mind the difference between specification vs. implementation!
 - Abstract Data Type: a type being specified
 - E.g. a class MyVector
 - Concrete Data Structure: how we implement a data type
 - E.g. how the MyVector class internally stores its data, by using arrays, or linked lists, etc.

Trees



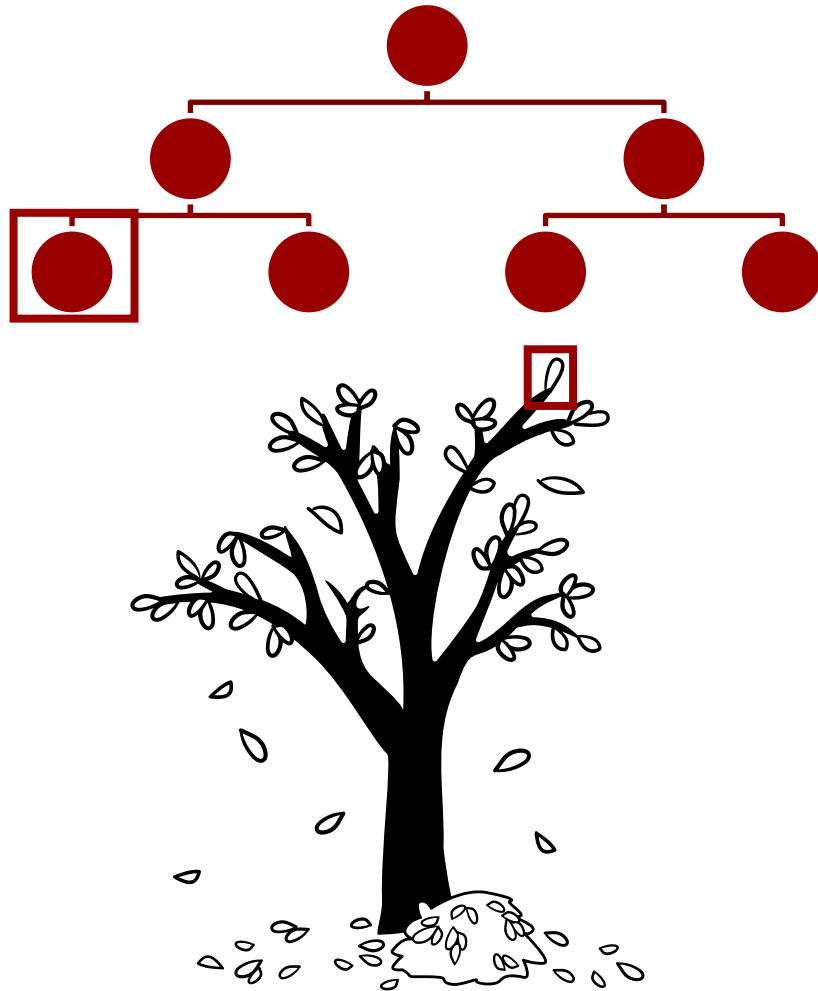
- Data structures useful to represent hierarchical dependencies
 - Root node: topmost node of a tree
 - Has no parent node
 - Has at least one child node

Trees



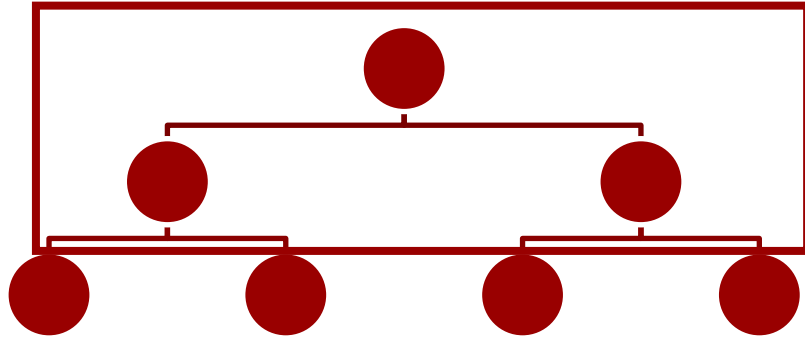
- Data structures useful to represent hierarchical dependencies
 - Root node: topmost node of a tree
 - Has no parent node
 - Has at least one child node
 - Intermediate node:
 - Has one parent node
 - Has one or more child nodes

Trees



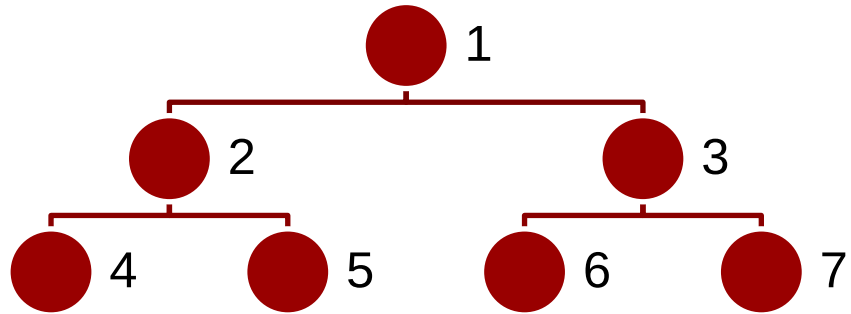
- Data structures useful to represent hierarchical dependencies
 - Root node: topmost node of a tree
 - Has no parent node
 - Has at least one child node
 - Intermediate node:
 - Has one parent node
 - Has one or more child nodes
 - Leaf node:
 - Has one parent node
 - Has no child nodes

Trees



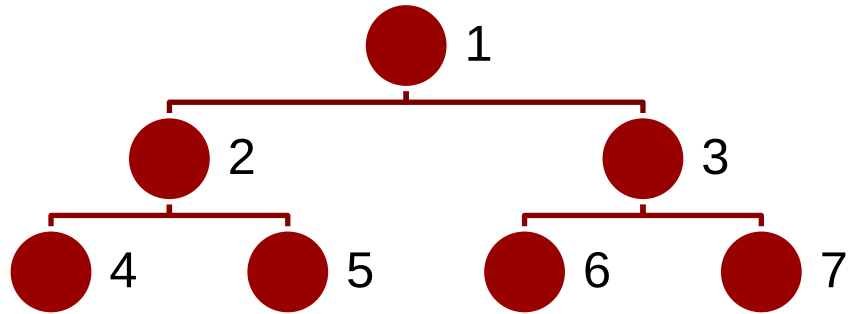
- Internal nodes:
 - Root node + intermediate nodes

Trees



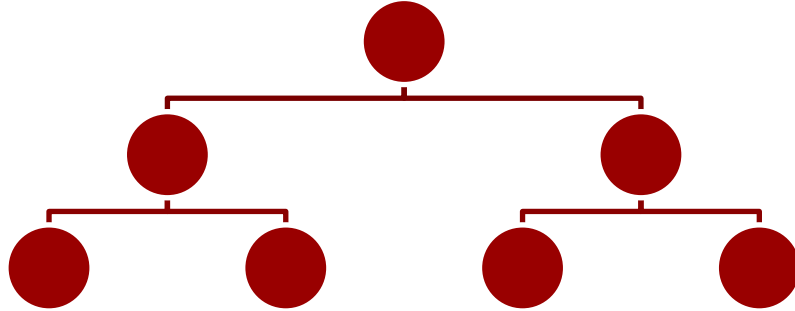
- Internal nodes:
 - Root node + intermediate nodes
- Size of a tree:
 - Number of nodes in a tree

Trees



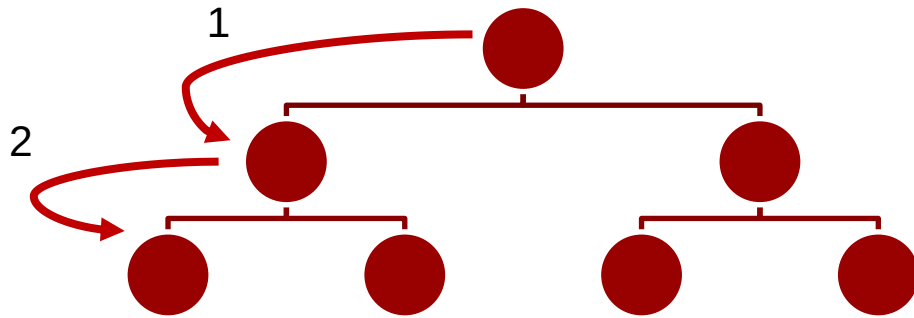
- Internal nodes:
 - Root node + intermediate nodes
- Size of a tree:
 - Number of nodes in a tree
 - In this example, size is 7

Trees



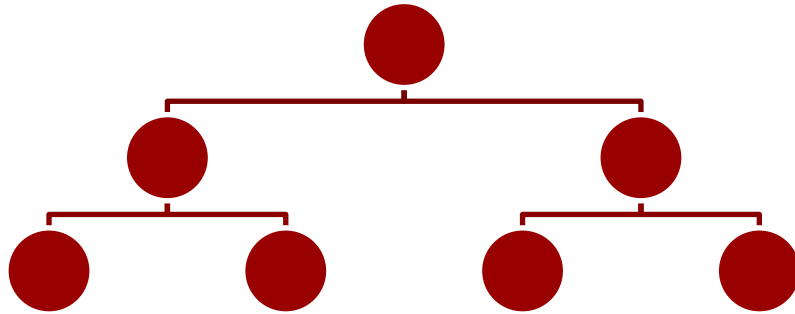
- Internal nodes:
 - Root node + intermediate nodes
- Size of a tree:
 - Number of nodes in a tree
 - In this example, size is 7
- Height of a tree:
 - Length (number of edges) of the longest path from the root to a leaf

Trees



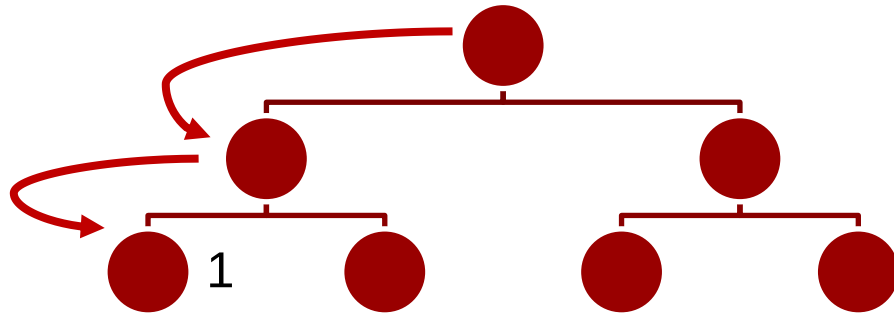
- Internal nodes:
 - Root node + intermediate nodes
- Size of a tree:
 - Number of nodes in a tree
 - In this example, size is 7
- Height of a tree:
 - Length (number of edges) of the longest path from the root to a leaf
 - In this example, size is 2

Trees



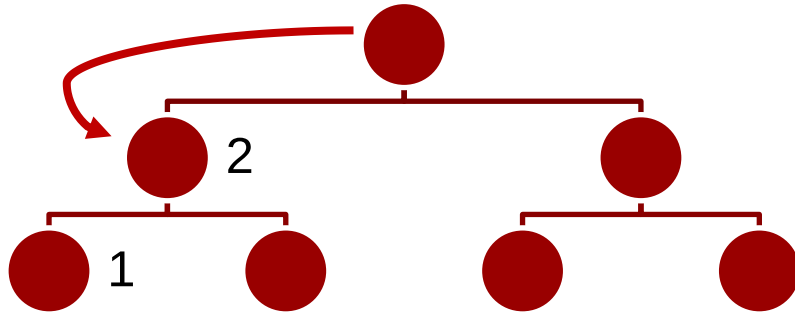
- Internal nodes:
 - Root node + intermediate nodes
- Size of a tree:
 - Number of nodes in a tree
 - In this example, size is 7
- Height of a tree:
 - Length (number of edges) of the longest path from the root to a leaf
 - In this example, size is 2
- Binary tree:
 - Tree whose nodes (except leaf nodes) have exactly 2 child nodes

Trees



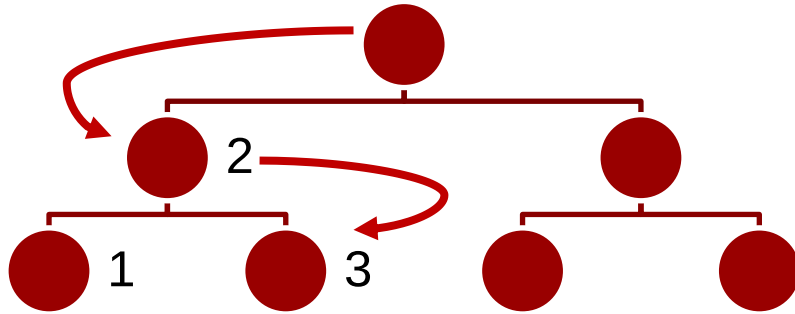
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



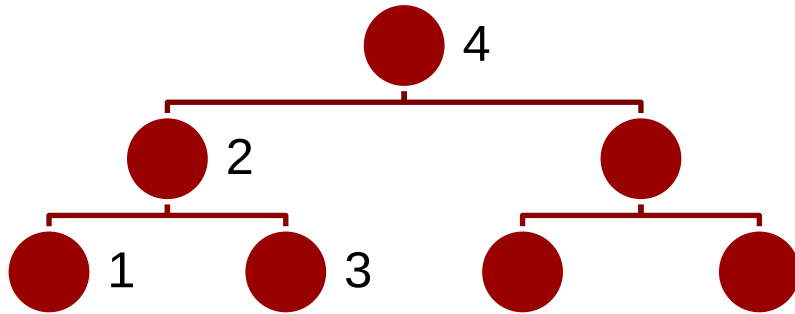
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



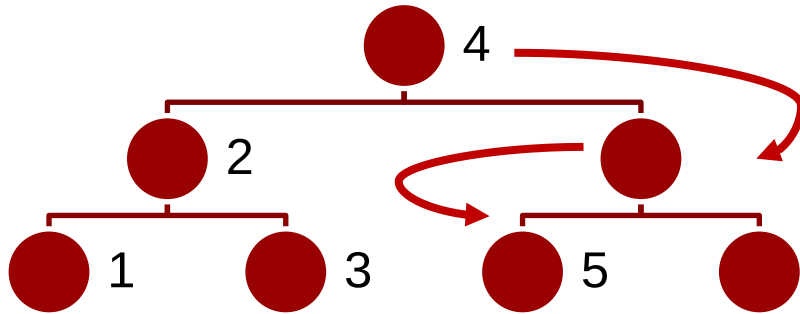
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



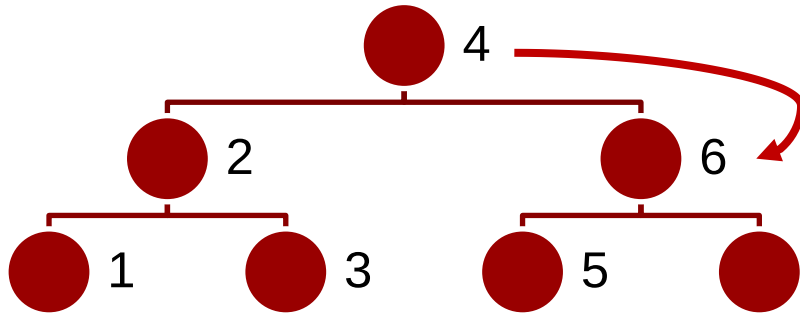
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



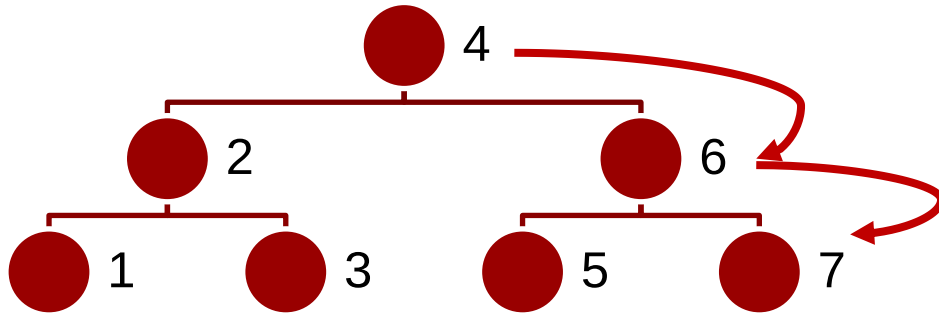
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



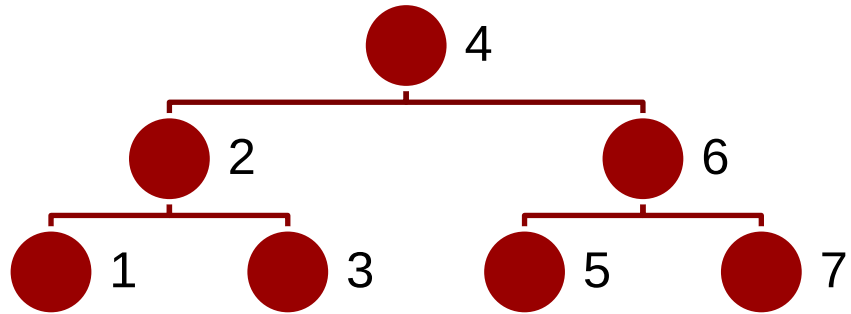
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



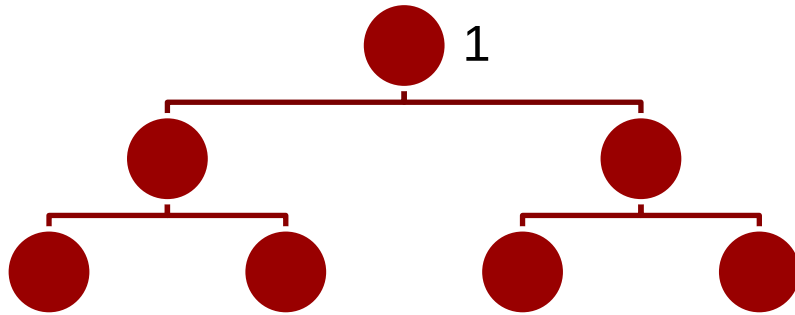
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



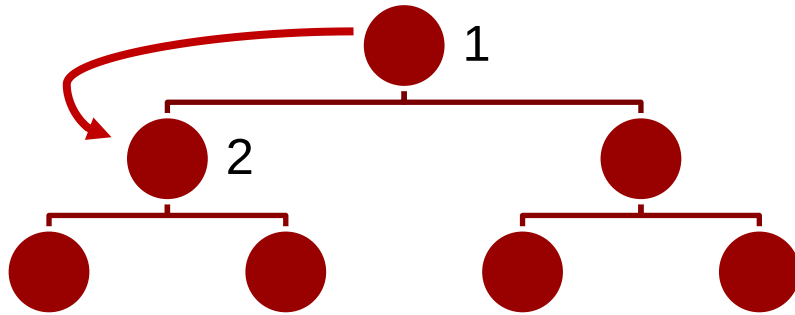
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent

Trees



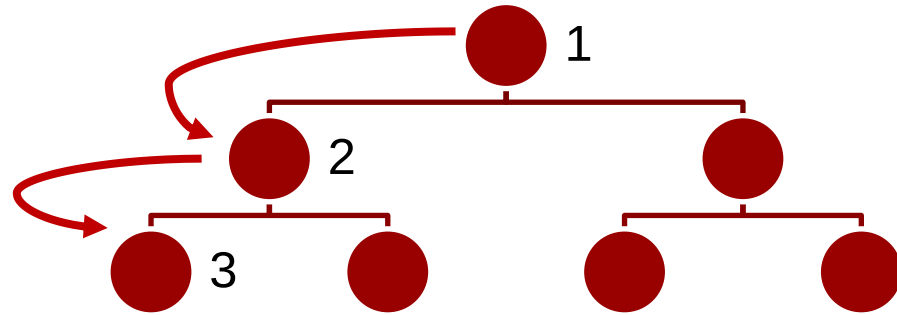
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



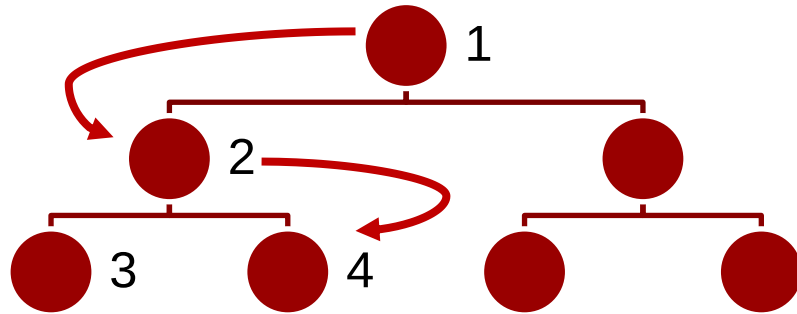
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



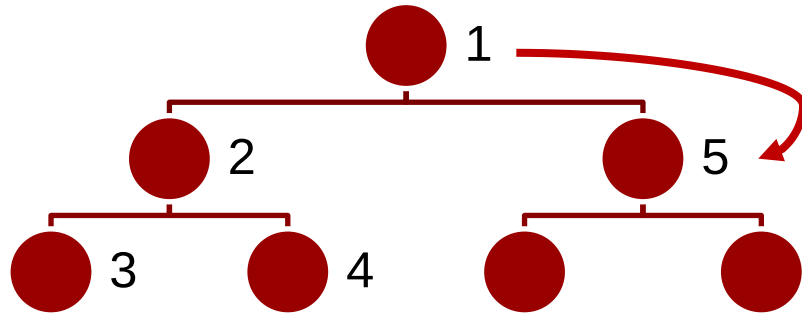
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



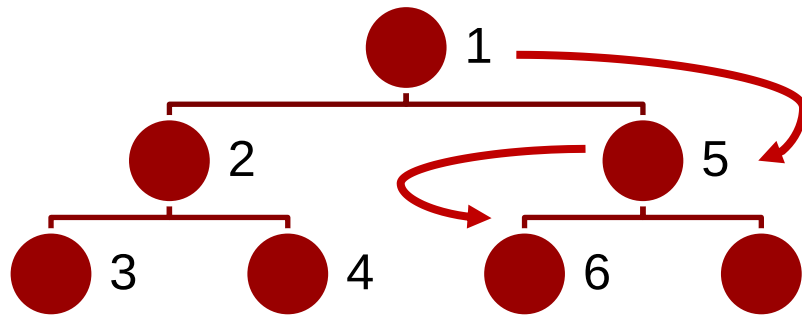
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



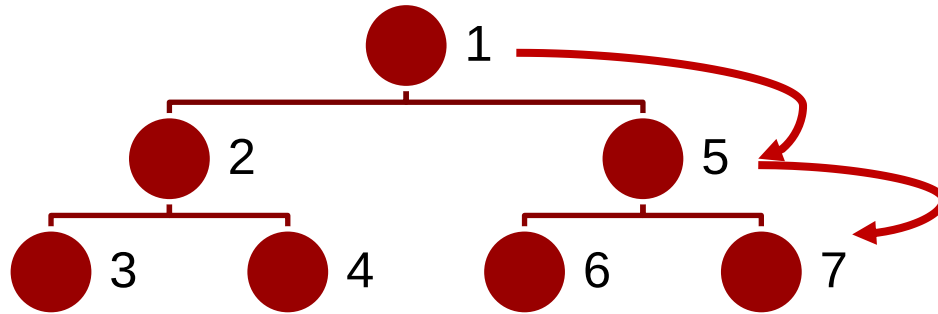
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



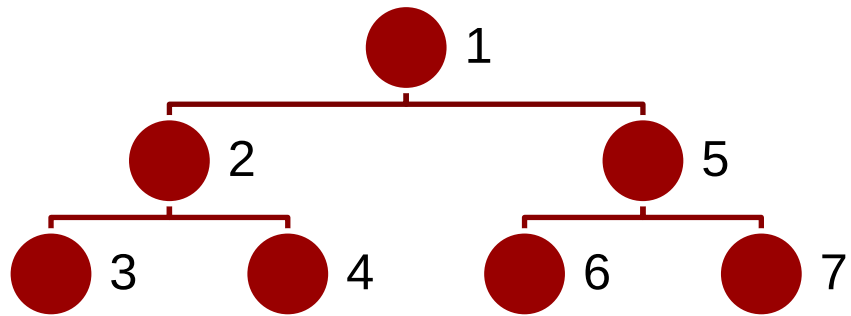
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



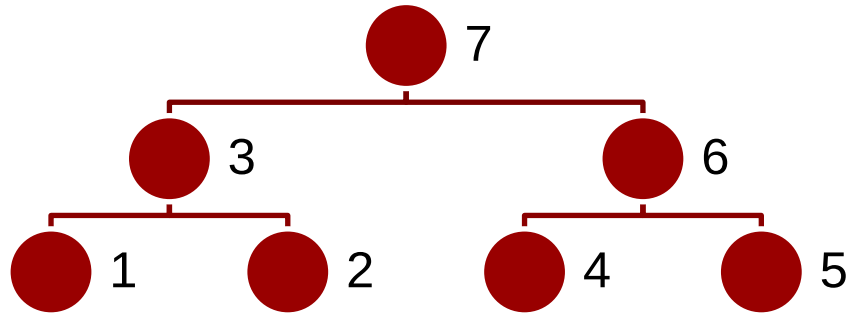
- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent

Trees



- In-order traversal:
 - Traverse the left descendent
 - Show the current node contents
 - Traverse the right descendent
- Pre-order traversal:
 - Show the current node contents
 - Traverse the left descendent
 - Traverse the right descendent
- Post-order traversal:
 - Traverse the left descendent
 - Traverse the right descendent
 - Show the current node contents

Representing trees in C++

```
1 enum NodeType { Internal, Leaf };
2
3 struct Node {
4     NodeType type;
5     int value; // Only used if type == Leaf
6     Node *left; // Only used if type != Leaf
7     Node *right; // Only used if type != Leaf
8 }
9
10 class Node {
11 public:
12     //methods to handle the tree
13
14 private:
15     NodeType type;
16     int value; // Only used if type == Leaf
17     Node *left; // Only used if type != Leaf
18     Node *right; // Only used if type != Leaf
19 }
```

- Similar to a single-linked list:
 - Each node is represented with a struct or a class

Representing trees in C++

```
1 enum NodeType { Internal, Leaf };
2
3 struct Node {
4     NodeType type;
5     int value; // Only used if type == Leaf
6     Node *left; // Only used if type != Leaf
7     Node *right; // Only used if type != Leaf
8 }
9
10 class Node {
11 public:
12     //methods to handle the tree
13
14 private:
15     NodeType type;
16     int value; // Only used if type == Leaf
17     Node *left; // Only used if type != Leaf
18     Node *right; // Only used if type != Leaf
19 }
```

- Similar to a single-linked list:
 - Each node is represented with a struct or a class
 - Each node has two pointers to the child nodes

Representing trees in C++

```
1 enum NodeType { Internal, Leaf };
2
3 struct Node {
4     NodeType type;
5     int value; // Only used if type == Leaf
6     Node *left; // Only used if type != Leaf
7     Node *right; // Only used if type != Leaf
8 }
9
10 class Node {
11 public:
12     //methods to handle the tree
13
14 private:
15     NodeType type;
16     int value; // Only used if type == Leaf
17     Node *left; // Only used if type != Leaf
18     Node *right; // Only used if type != Leaf
19 }
```

- Similar to a single-linked list:
 - Each node is represented with a struct or a class
 - Each node has two pointers to the child nodes
 - (optional) Each node is identified as leaf or internal node
 - Alternatively, to identify leaf nodes, one needs to set both pointers to nullptr

Computing the size of a tree in C++

```
1 enum NodeType { Internal, Leaf };
2
3 struct Node {
4     NodeType type;
5     int value; // Only used if type == Leaf
6     Node *left; // Only used if type != Leaf
7     Node *right; // Only used if type != Leaf
8 }
9
10 int size(Node *node) {
11     if (node->type == Leaf) {
12         return 1;
13     } else {
14         return 1 + size(node->left) + size(node->right);
15     }
16 }
```

- Current node is a leaf node (base case)
 - Size is 1

Computing the size of a tree in C++

```
1 enum NodeType { Internal, Leaf };
2
3 struct Node {
4     NodeType type;
5     int value; // Only used if type == Leaf
6     Node *left; // Only used if type != Leaf
7     Node *right; // Only used if type != Leaf
8 }
9
10 int size(Node *node) {
11     if (node->type == Leaf) {
12         return 1;
13     } else {
14         return 1 + size(node->left) + size(node->right);
15     }
16 }
```

- Current node is a leaf node (base case)
 - Size is 1
- Current node is an internal node:
 - Size is 1 + sizes of its child nodes

Computing the size of a tree in C++

```
1 enum NodeType { Internal, Leaf };
2
3 struct Node {
4     NodeType type;
5     int value; // Only used if type == Leaf
6     Node *left; // Only used if type != Leaf
7     Node *right; // Only used if type != Leaf
8 }
9
10 int size(Node *node) {
11     if (node->type == Leaf) {
12         return 1;
13     } else {
14         return 1 + size(node->left) + size(node->right);
15     }
16 }
```

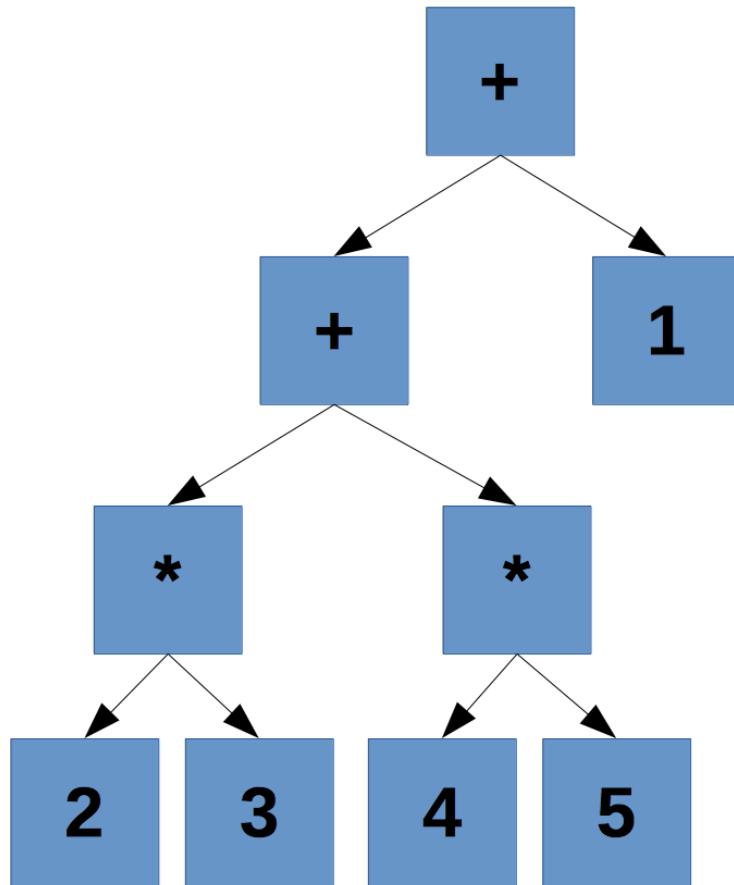
- Current node is a leaf node (base case)
 - Size is 1
- Current node is an internal node:
 - Size is 1 + sizes of its child nodes
- Non-recursive implementations are possible, but more complicated!

Using trees to represent expressions

- How can we represent the following expression?

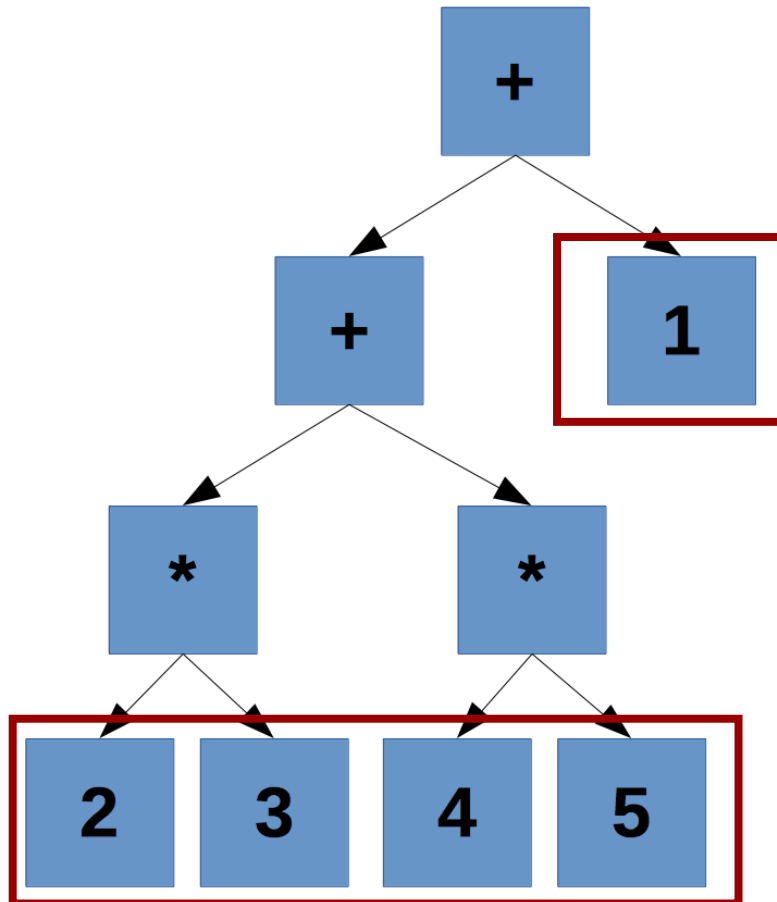
$$(2 * 3) + (4 * 5)) + 1$$

Using trees to represent expressions



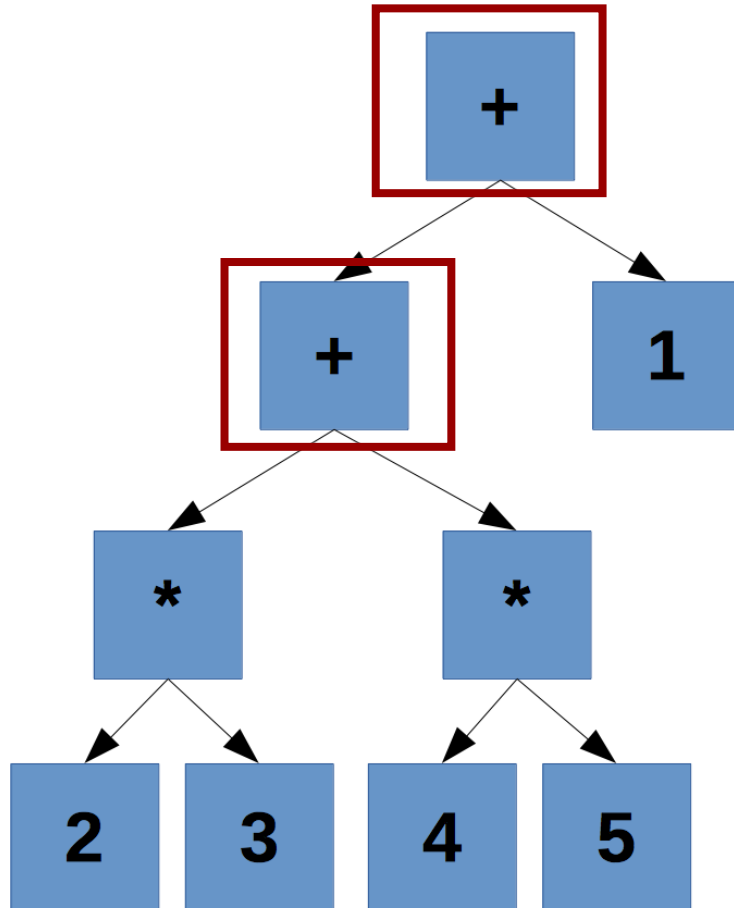
- How can we represent the following expression?
 $(2 * 3) + (4 * 5) + 1$
- We can use a binary tree!

Using trees to represent expressions



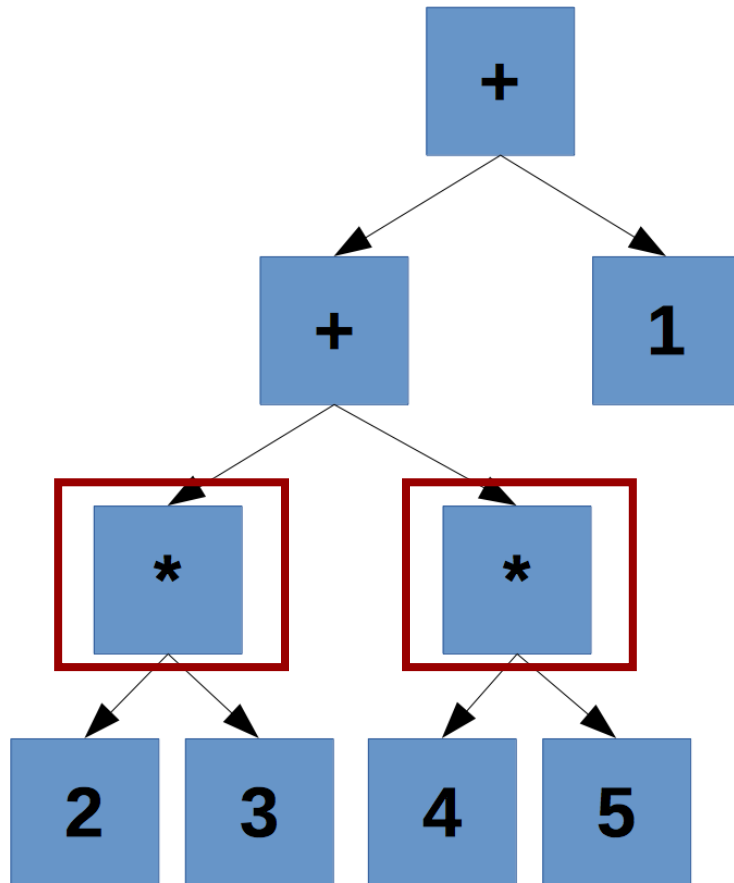
- How can we represent the following expression?
 $(2 * 3) + (4 * 5) + 1$
- We can use a binary tree!
- We need to represent:
 - Constants

Using trees to represent expressions



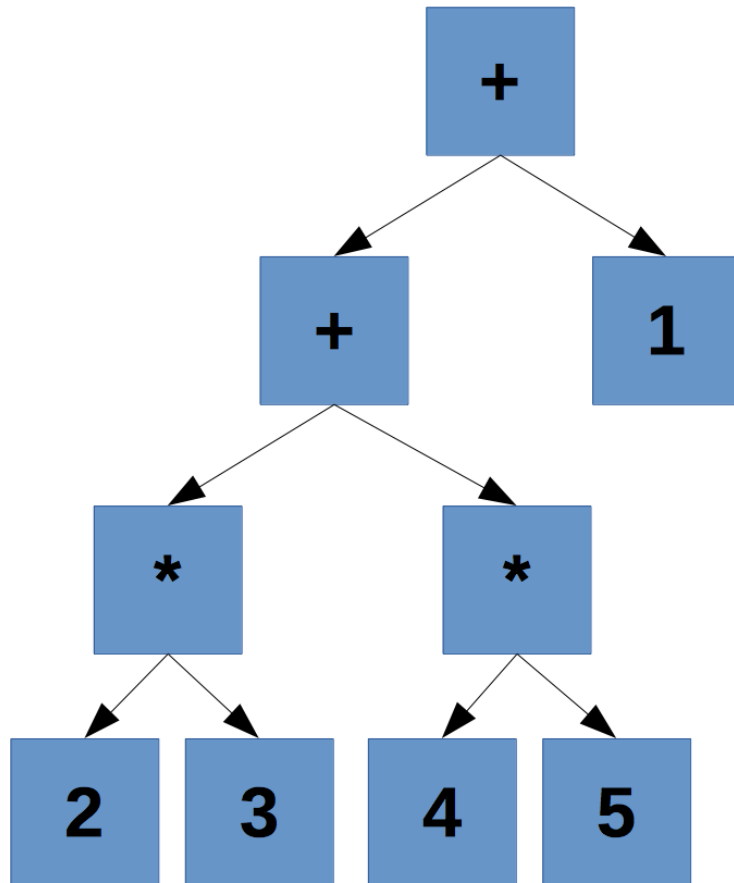
- How can we represent the following expression?
$$(2 * 3) + (4 * 5) + 1$$
- We can use a binary tree!
- We need to represent:
 - Constants
 - The sum of two arithmetic expressions

Using trees to represent expressions



- How can we represent the following expression?
 $(2 * 3) + (4 * 5) + 1$
- We can use a binary tree!
- We need to represent:
 - Constants
 - The sum of two arithmetic expressions
 - The product of two arithmetic expressions

Using trees to represent expressions



- During live coding session, we will see how to:
 - Compute size
 - Compute height
 - Compute number of leaves
 - Traverse the tree in different ways:
 - In-order: print algebraic notation
 - Pre-order
 - Post-order: print reverse polish notation

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 11
 - Ask questions if something is unclear (including previous assignments)
 - Make sure you can run the local autograder for May 2021 and September 2021 exams

DTU

